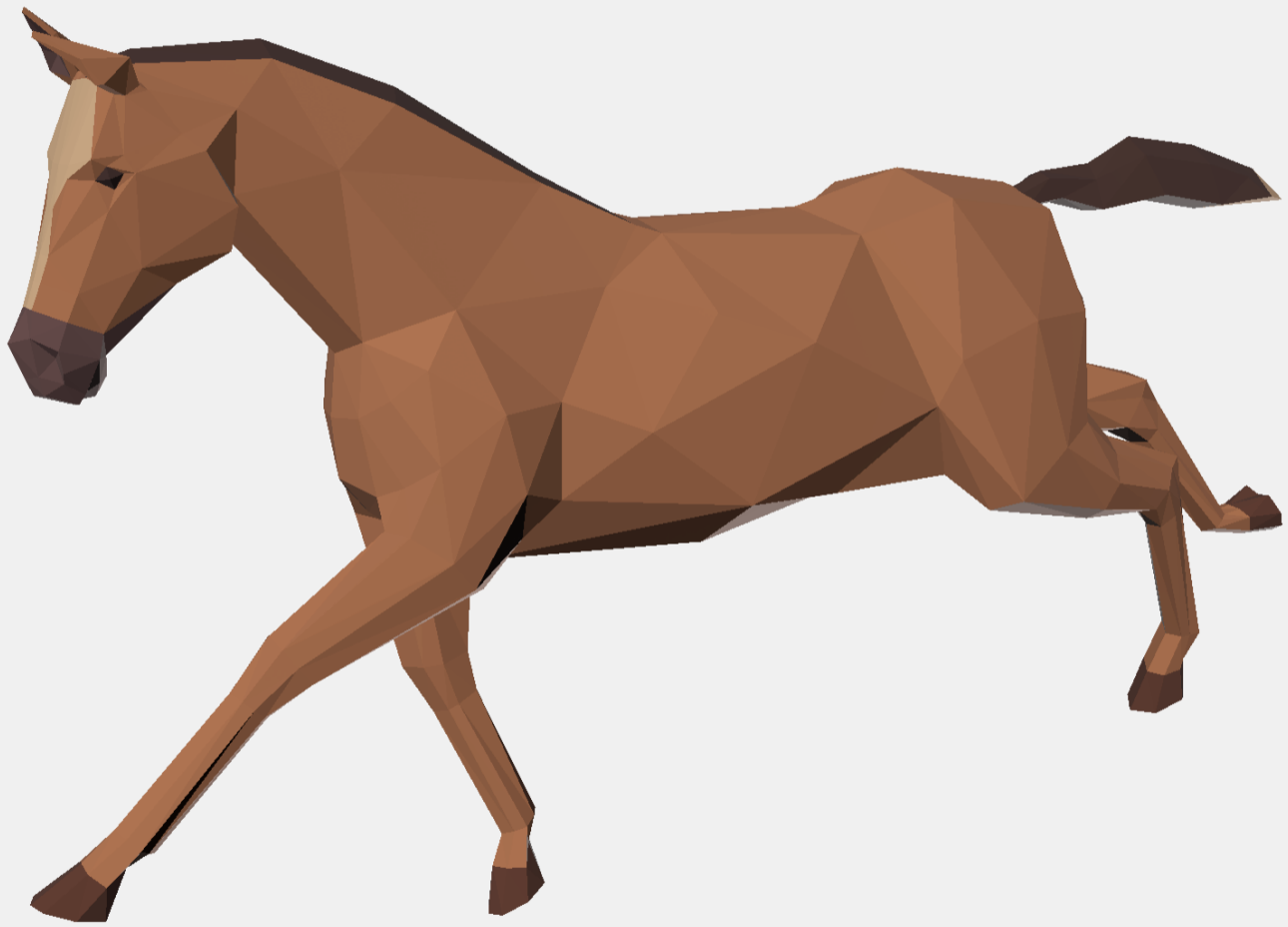


RTR – GPU Pipeline & OpenGL

David Schedl

University of Applied Sciences Upper Austria



Real-Time Rendering

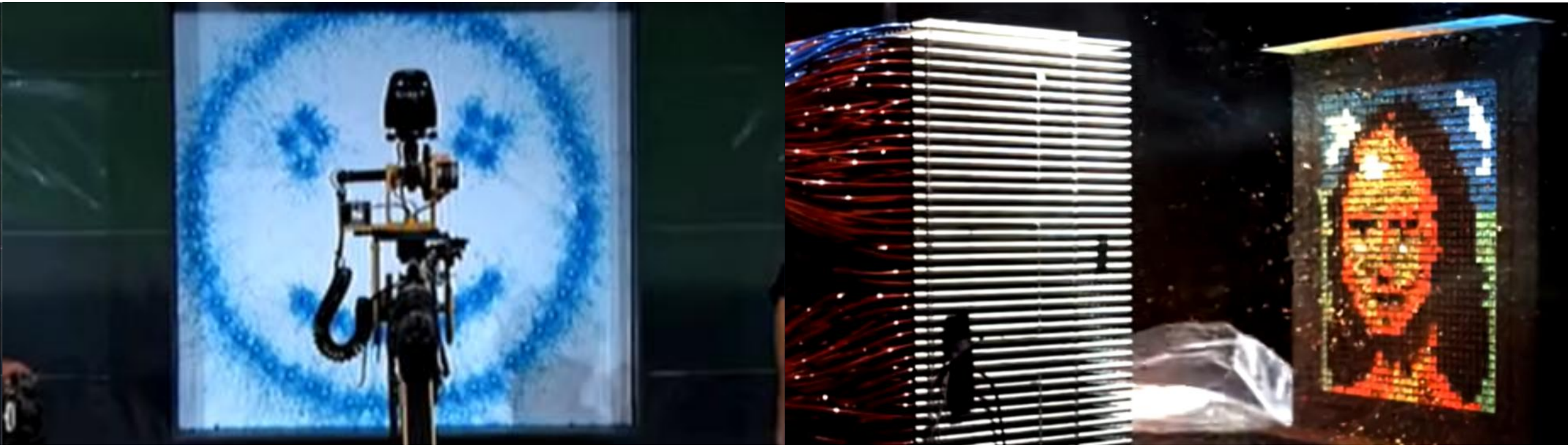


Horizon Zero Dawn (2017)

Motivation

CPU (sequential)

GPU (parallel)



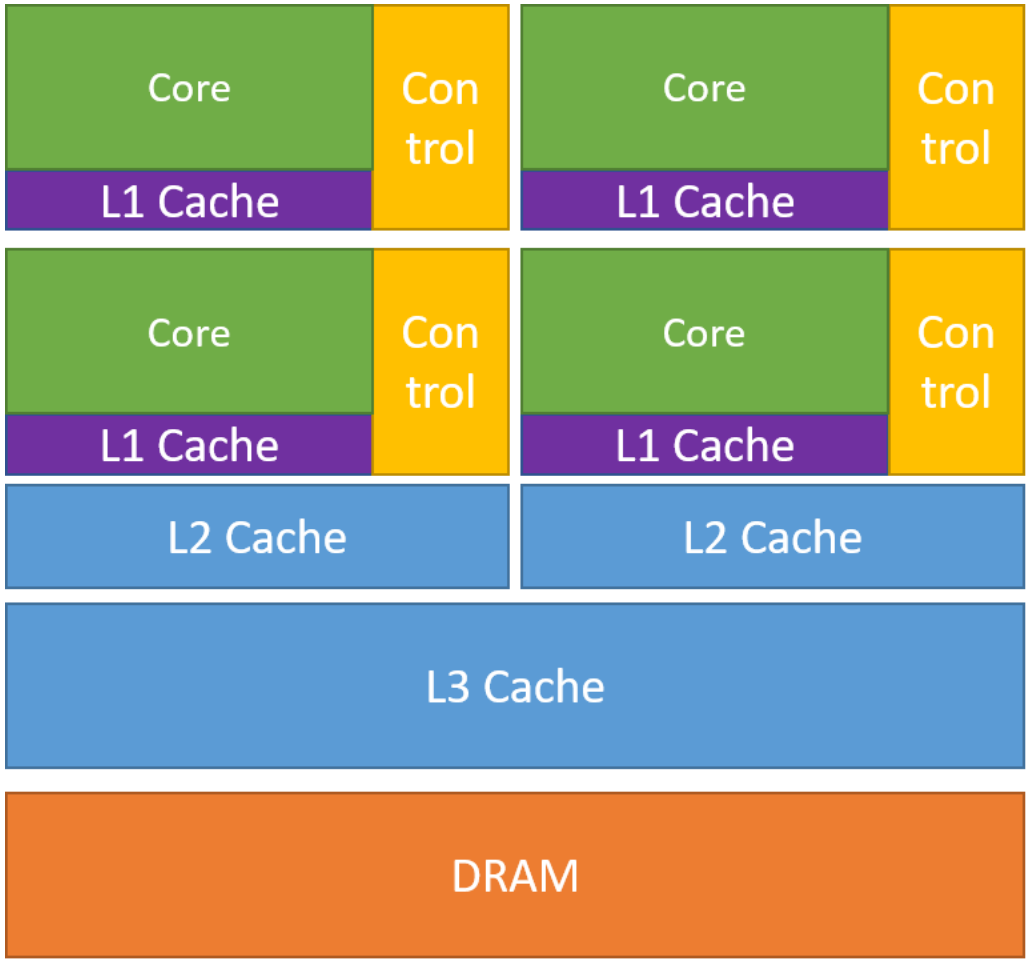
NVIDIA and MythBusters (2009)

https://www.youtube.com/watch?v=ZrJeYFxpUyQ

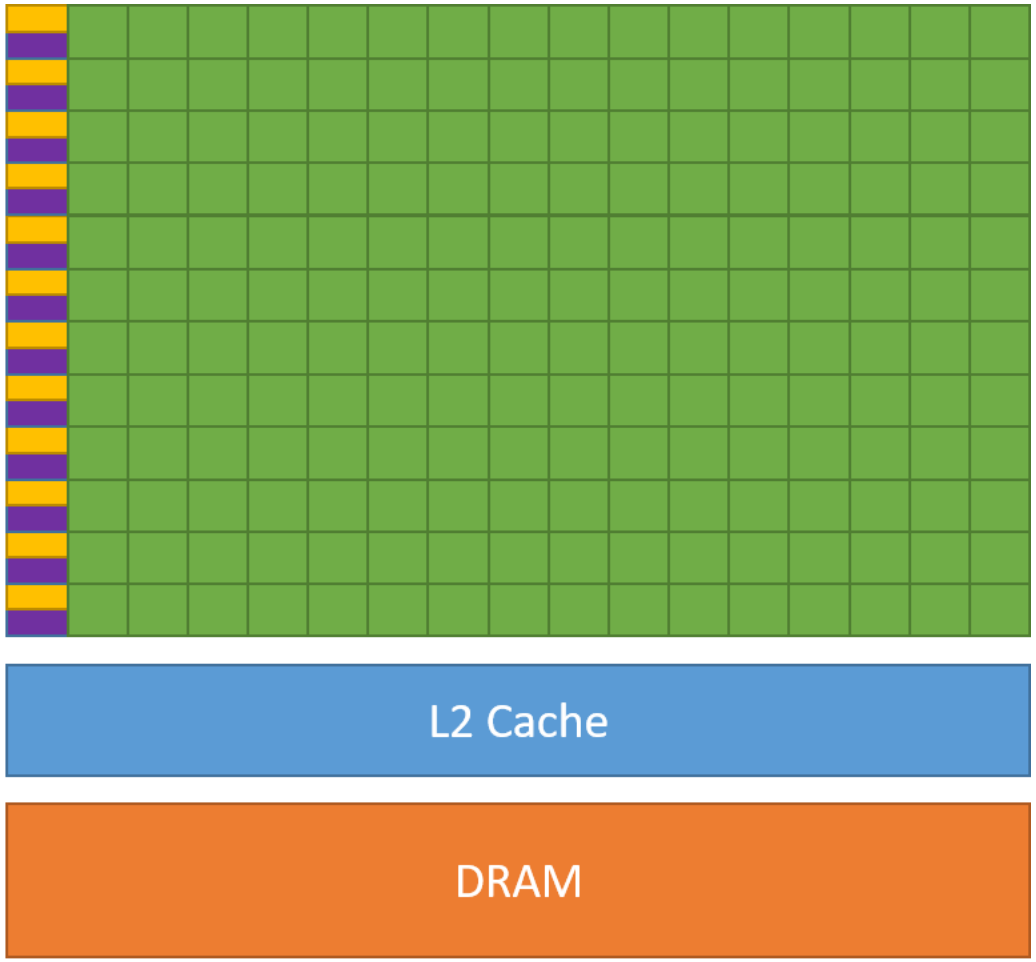
More about GPUs



CPU



GPU



https://docs.nvidia.com/cuda/cuda-c-programming-guide/index.html

More about GPUs

CPU

- Less cores/parallelism
- Versatile:
  - more cache
  - larger control structures

GPU

- Many cores
- Very specialized processing:
  - a few instructions for massive data (single instruction multiple data -> SIMD)
- Not well suited (slow) for:
  - branching (if/else, switch, varying loop iterations)
  - data dependencies or synchronization

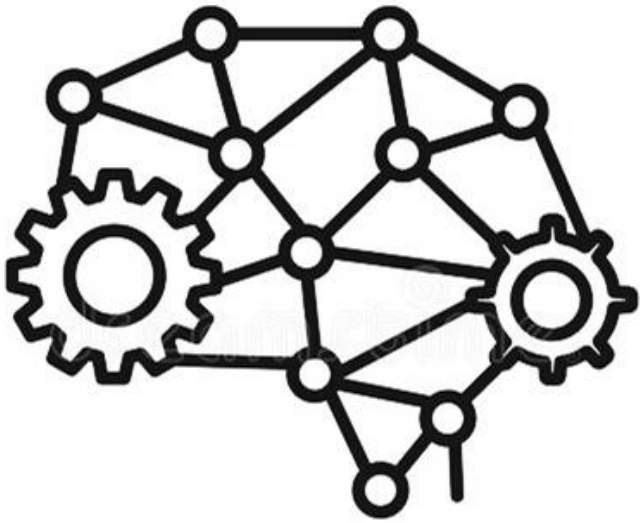
GPU Applications



Computer Graphics in Games and Movies



Crypto Mining



Deep Learning

OpenGL

- Open Graphics Library by the Khronos group (non-profit)
- Low-Level (Software) API for graphic programming
- Various bindings (C/C++, Pyhton, Java, …) and versions (WebGL, OGL ES, …)
- On desktop:
  - OpenGL comes with GPU drivers (Nvidia, AMD, …),
  - no window/image handling, no keyboard/input support, and no high-level support (e.g. texture or geometry loading)
- Think of OpenGL as a standard for talking to your GPU!



Versions



Desktop



Mobile or low-power devices



In Browser 

ES 2.0 ⟷ WebGL 1

ES 3.0 ⟷ WebGL 2

Alternatives:

Microsoft® DirectX®



History

OpenGL		OpenGL ES		Features Added
1.0	~1992			Fixed Pipeline
1.3	2001	ES 1.0	2003	
2.0	2004	ES 2.0	2007	+Programmable Pipeline
		ES 3.0	2012	
3.0	2008	ES 3.1	2014	+Geometry shaders
4.0	2010	ES 3.2	2015	+Tessellation shaders
4.6 (latest)	2017			

WebGL



- Is based on OpenGL ES 2.0 and ES 3.0 (WebGL 2).
- GPU-acceleration in browsers via HTML and JavaScript.
- Browser takes care of high-level tasks
  (displaying, user inputs, loading resources, …)

- Most browsers use **ANGLE: Almost Native Graphics Layer Engine**
  - OpenGL ES functions to other APIs
    (i.e., DirectX, OpenGL, Vulkan, or Metal)

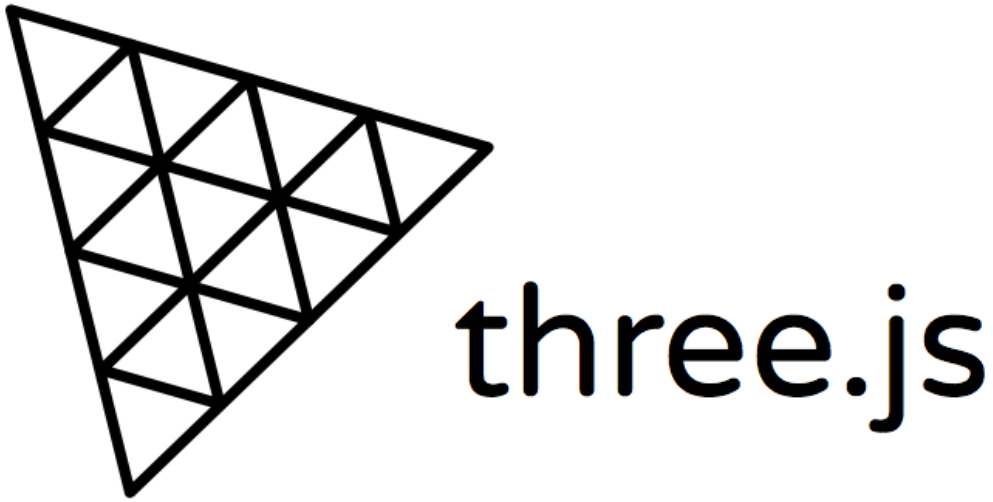




Well-known WebGL Frameworks



https://www.babylonjs.com/

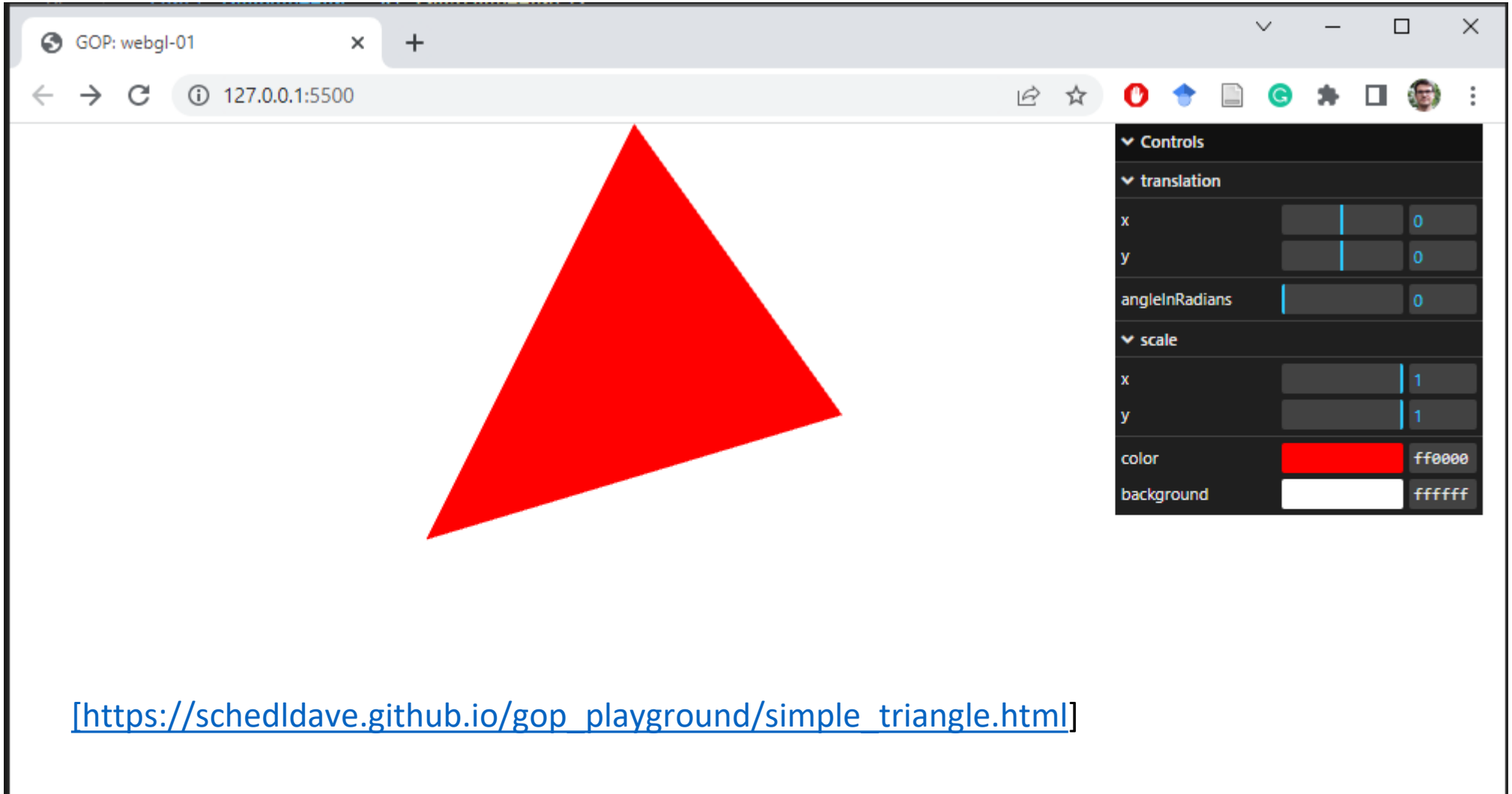


https://threejs.org/

Like our lab code!

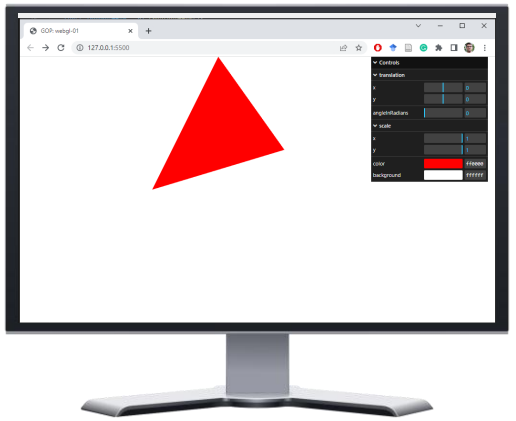
Let’s get started

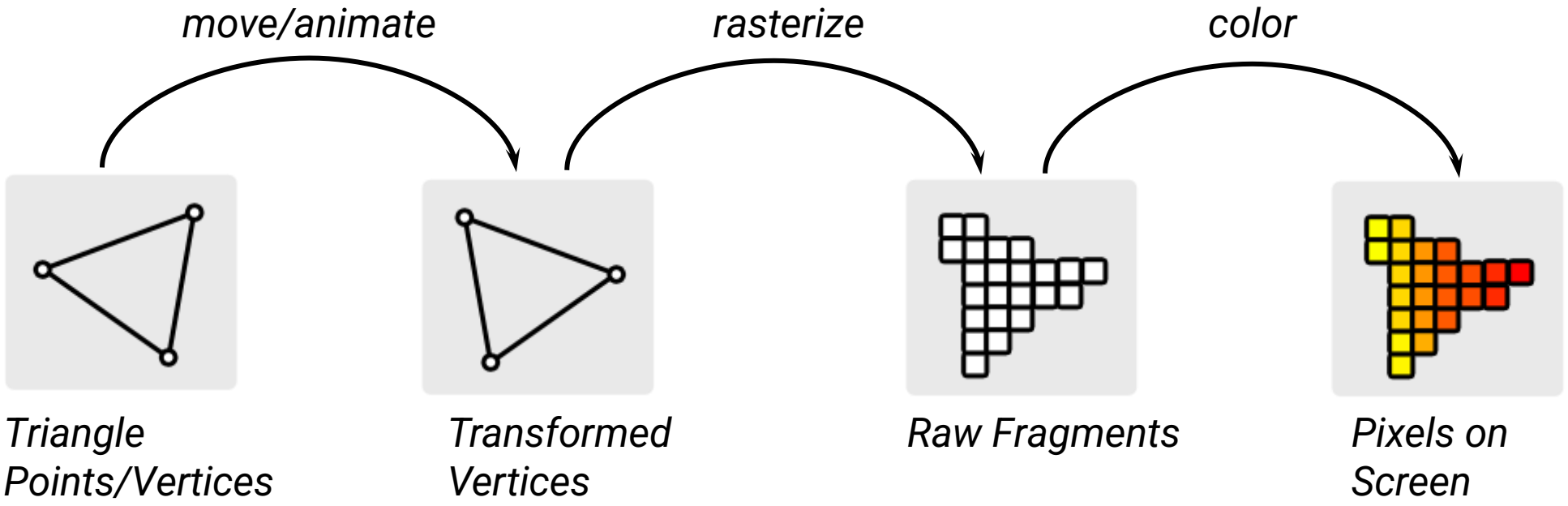
Hello ~~World~~ Triangle!



[https://schedldave.github.io/gop_playground/simple_triangle.html]

The Graphics Pipeline (on GPU) aka “What is really going on”






WebGL™

HTML Canvas

- The WebGL API uses the HTML `<canvas>` element.
- Context (`WebGL2RenderingContext`) provides OpenGL functions.
- We use a variable `gl` or `_gl` for the WebGL context.
- Only one context per canvas
  (no sharing between multiple canvases).

```html
<canvas id="canvas"></canvas>
```

HTML

```
const canvas = document.getElementById('canvas');
const gl = canvas.getContext('webgl2’);
// gl is of type WebGL2RenderingContext
```

JS

WebGL2RenderingContext

- We use a variable `gl` or `_gl` for the WebGL context.

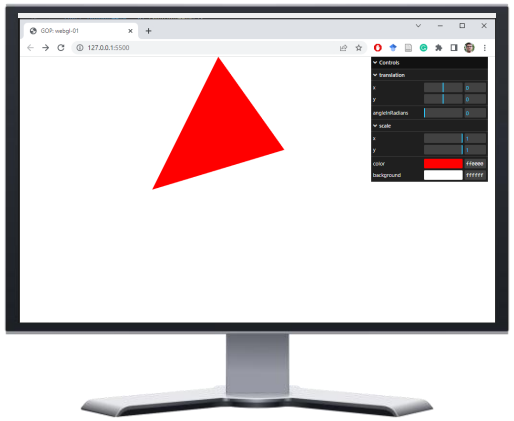
- Example functions to draw/change the canvas:
  - Setting up the viewport.
  - Clearing the canvas with a constant color.

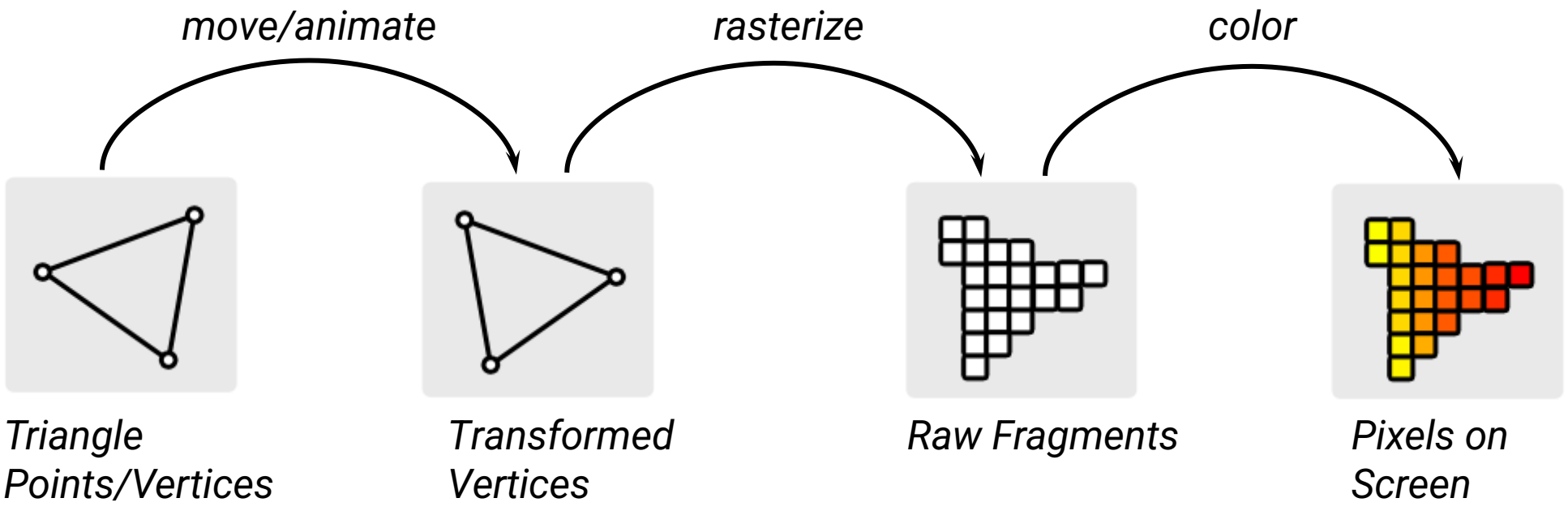
JS

```
// Define the size of the WebGL window/view on the canvas in pixel
// maps NDC to window coordinates
gl.viewport(0, 0, gl.canvas.width, gl.canvas.height);
// Clear the canvas
gl.clearColor(0, 0, 0, 0);
gl.clear(gl.COLOR_BUFFER_BIT | gl.DEPTH_BUFFER_BIT);
```

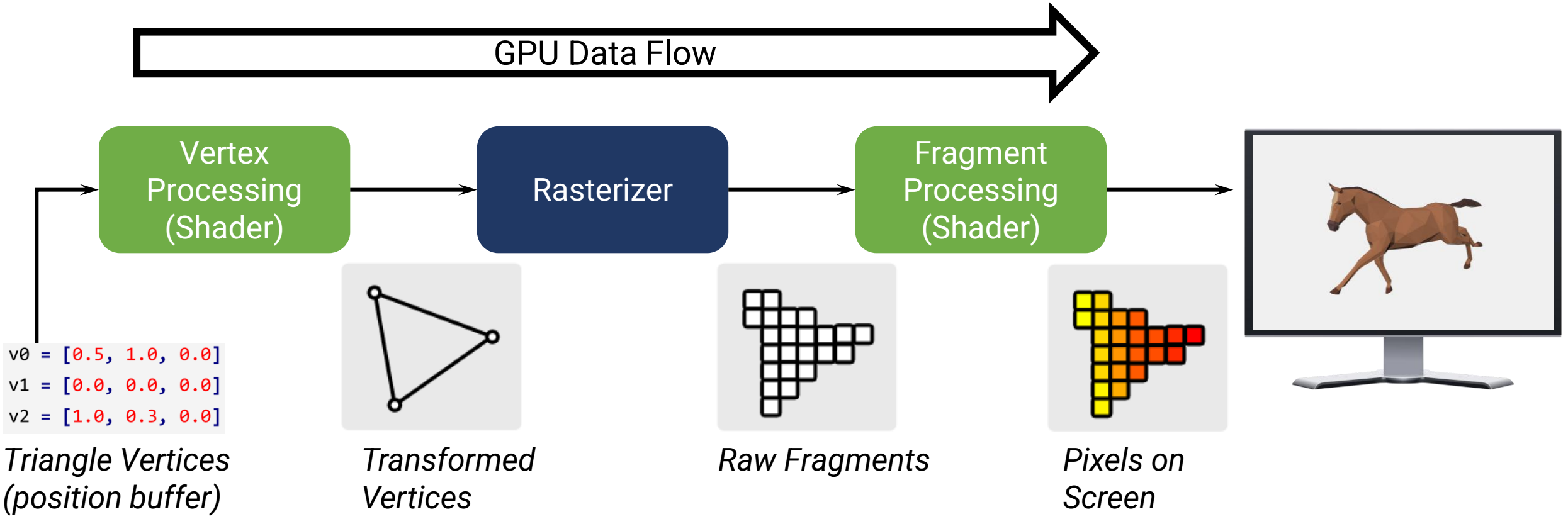
Color (0 to 1)

The Graphics Pipeline (on GPU)





The Programmable Pipeline



Shaders

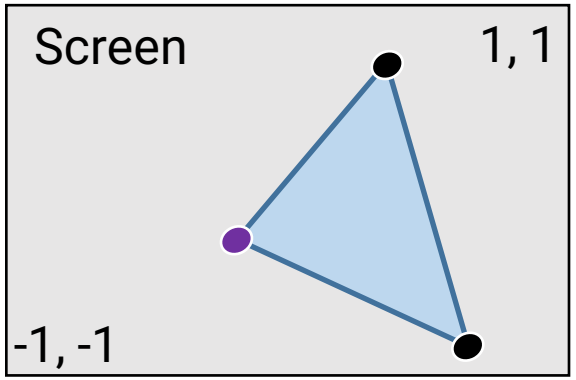
- Simple/Small programs running on the GPU.
- Needs to be compiled and loaded on the GPU with OpenGL.
- OpenGL Shading Language (GLSL) - syntax like C/C++ or Java.
- Specialized for Matrix and Vector operations:
  https://www.khronos.org/registry/OpenGL/specs/es/3.0/GLSL_ES_Specification_3.00.pdf

Vertex-Shader **GLSL**

```
#version 300 es
layout(location=0) in vec3 position; // 3D Vector
uniform mat4 modelMatrix; // 4x4 Matrix
void main() {
  gl_Position = modelMatrix * vec4( position, 1.0 );
}
```

always only one main!

Vertex Shader



- Processes one vertex (position) of a triangle.
- Computes the *Transformed Vertex* position (`gl_Position`).
  - A position “in view” needs to be in -1 to 1 for x,y,z (yes z too!); called normalized device coordinates (NDC) or clip space
    ➔ everything outside -1 to 1 is clipped (not visible)

Vertex-Shader **GLSL**

```
#version 300 es

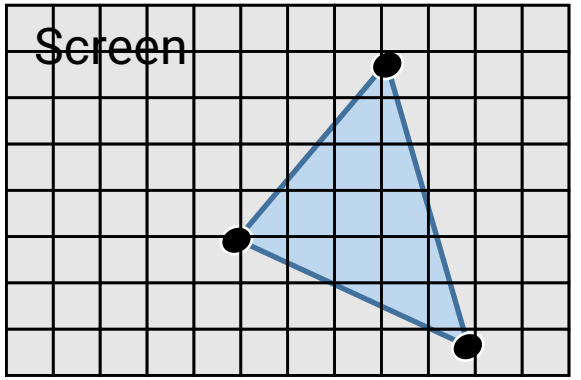
layout(location=0) in vec3 position; // 3D Vector

uniform mat4 modelMatrix; // 4x4 Matrix

void main() {
  gl_Position = modelMatrix * vec4( position, 1.0 ); // compute NDC
}
```

Transformed Vertex position

Programmable Pipeline



GPU Data Flow

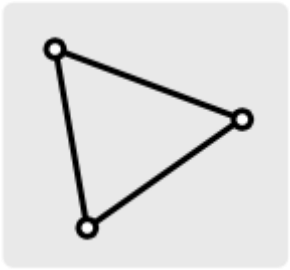
Vertex Processing (Shader)

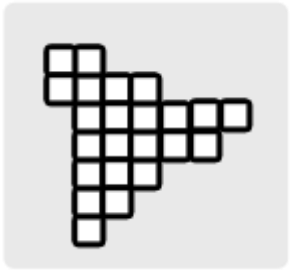
Rasterizer

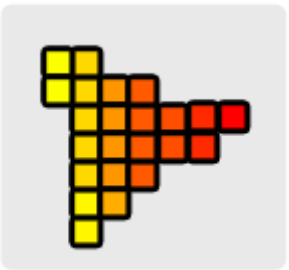
Fragment Processing (Shader)



```
v0 = [0.5, 1.0, 0.0]
v1 = [0.0, 0.0, 0.0]
v2 = [1.0, 0.3, 0.0]
```







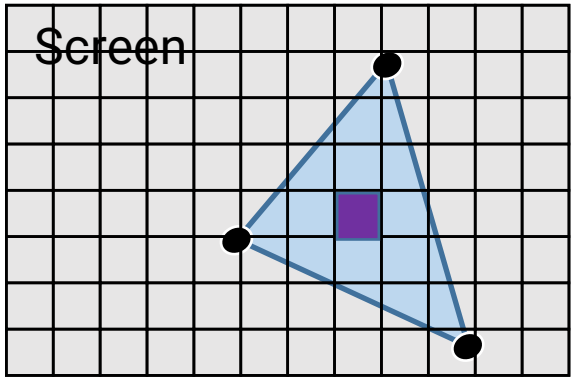
Triangle Vertices (position buffer)

Transformed Vertices (NDC)

Raw Fragments

Pixels on Screen

Fragment Shader

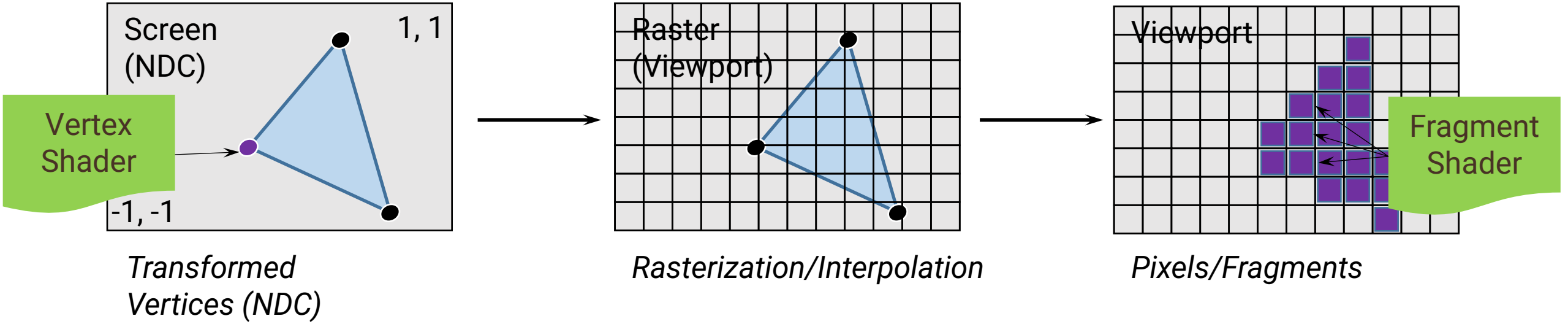


- Processes one fragment of a rasterized triangle.
  - Note: there might be many more fragments than vertices!
- Computes the *Pixel/Fragment Color* (`out` variable).
- Can get quite complex (i.e., https://www.shadertoy.com/user/iq/)!

Fragment-Shader **GLSL**

```
#version 300 es
precision highp float;
uniform vec3 color; // color input
out vec4 fragColor; // fragment color
void main() {
    fragColor = vec4( color, 1.0 ); // only assign fragment color
}
```

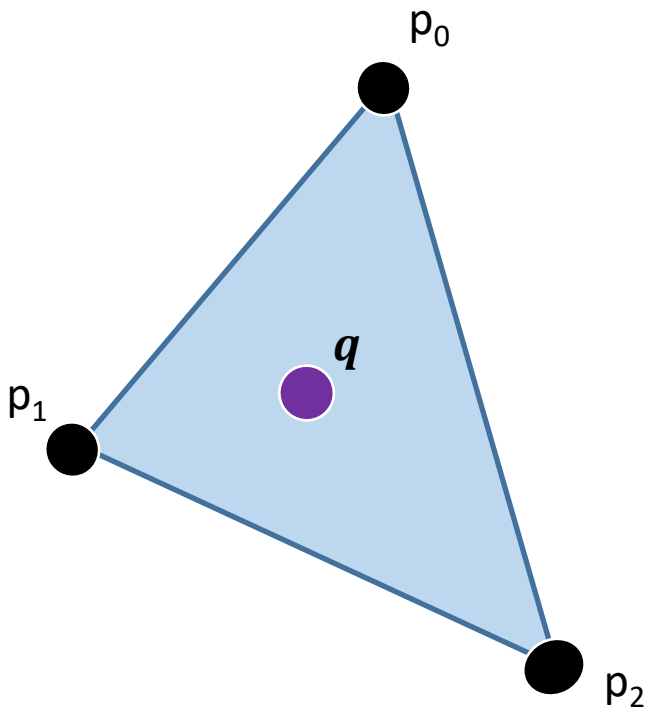
Raster Graphics



Rasterizer calculates:

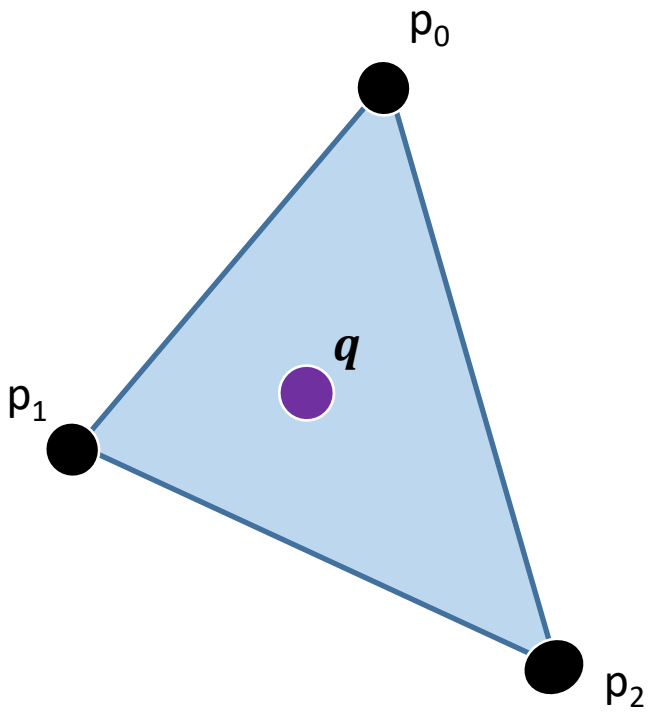
- If fragment is inside a triangle.
- Positions (and out variables/attributes) in the triangle.

Triangle



$$\boldsymbol{q} = \alpha \boldsymbol{p}_0 + \beta \boldsymbol{p}_1 + \gamma \boldsymbol{p}_2$$

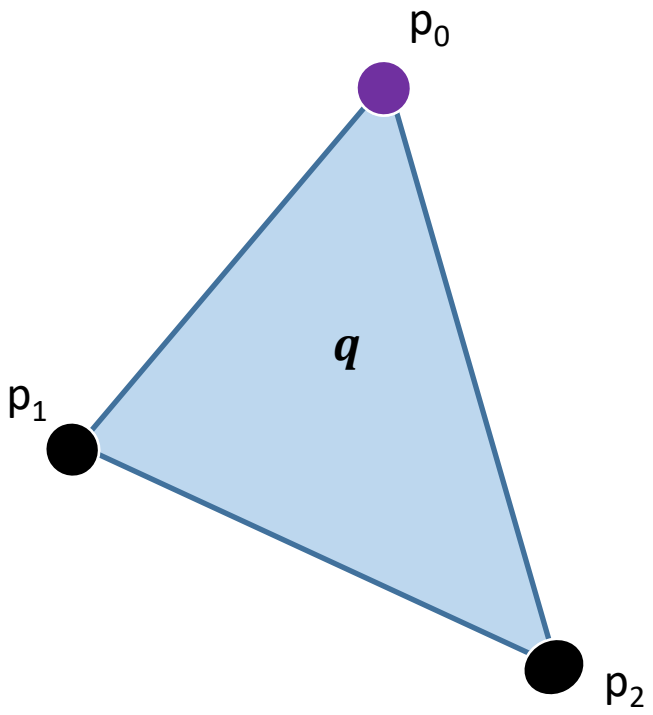
Barycentric Coordinates



$$\boldsymbol{q} = \alpha \boldsymbol{p}_0 + \beta \boldsymbol{p}_1 + \gamma \boldsymbol{p}_2$$

$$\alpha + \beta + \gamma = 1$$

Barycentric Coordinates

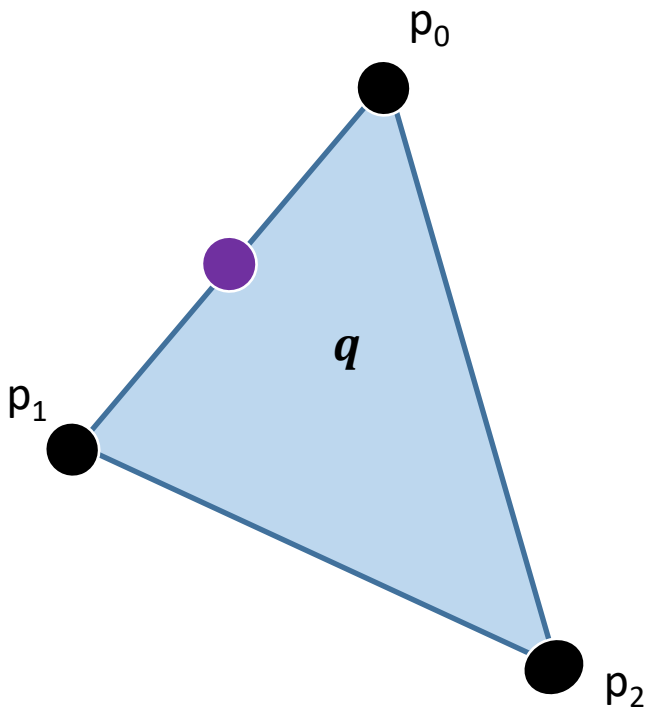


$$\boldsymbol{q} = \alpha \boldsymbol{p}_0 + \beta \boldsymbol{p}_1 + \gamma \boldsymbol{p}_2$$

$$\alpha + \beta + \gamma = 1$$

$$\alpha = 1 \quad \beta = 0 \quad \gamma = 0$$

Barycentric Coordinates

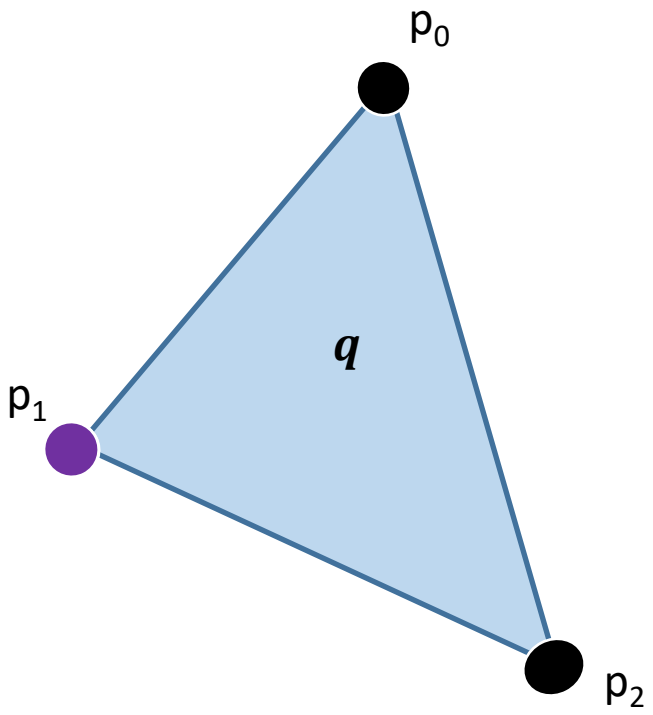


$$\boldsymbol{q} = \alpha \boldsymbol{p}_0 + \beta \boldsymbol{p}_1 + \gamma \boldsymbol{p}_2$$

$$\alpha + \beta + \gamma = 1$$

$$\alpha = 0.5 \quad \beta = 0.5 \quad \gamma = 0$$

Barycentric Coordinates

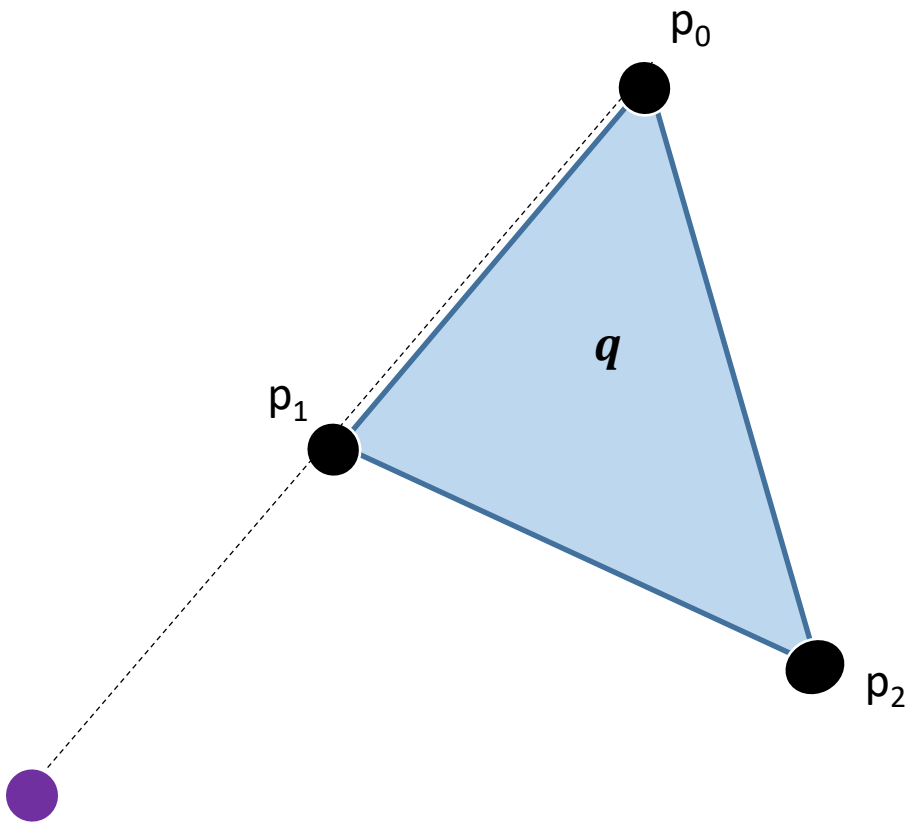


$$\boldsymbol{q} = \alpha \boldsymbol{p}_0 + \beta \boldsymbol{p}_1 + \gamma \boldsymbol{p}_2$$

$$\alpha + \beta + \gamma = 1$$

$$\alpha = 0 \quad \beta = 1 \quad \gamma = 0$$

Barycentric Coordinates

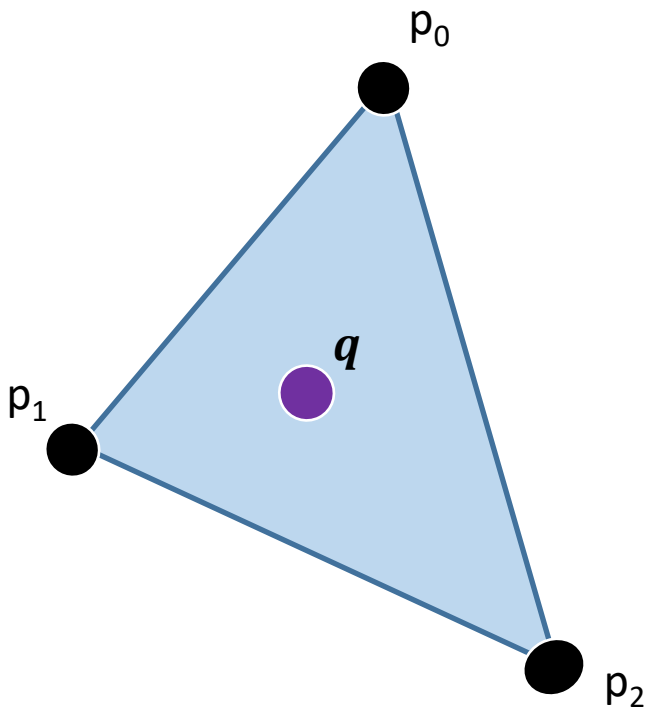


$$\boldsymbol{q} = \alpha \boldsymbol{p}_0 + \beta \boldsymbol{p}_1 + \gamma \boldsymbol{p}_2$$

$$\alpha + \beta + \gamma = 1$$

$$\alpha = -1 \quad \beta = 2 \quad \gamma = 0$$

Barycentric Coordinates



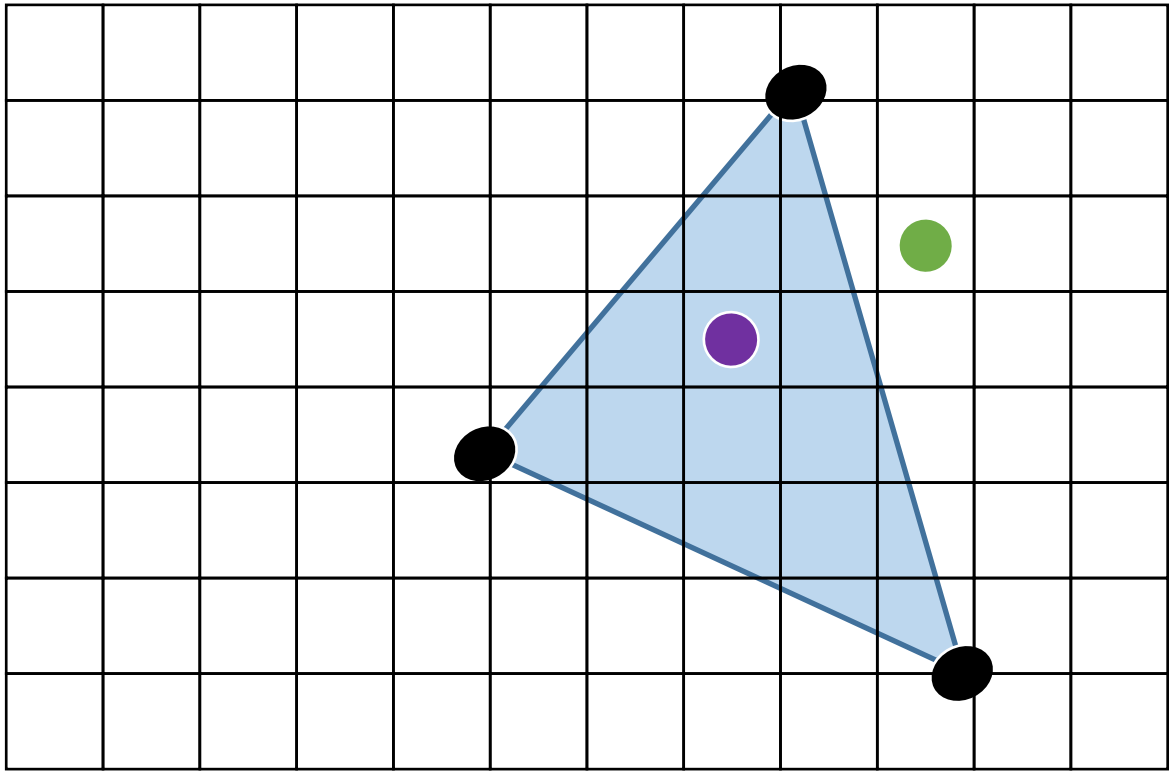
$$\boldsymbol{q} = \alpha \boldsymbol{p}_0 + \beta \boldsymbol{p}_1 + \gamma \boldsymbol{p}_2$$

$$\alpha + \beta + \gamma = 1$$

Inside triangle if:

$$0 \leq \alpha, \beta, \gamma \leq 1$$

Rasterization: Test if inside or outside



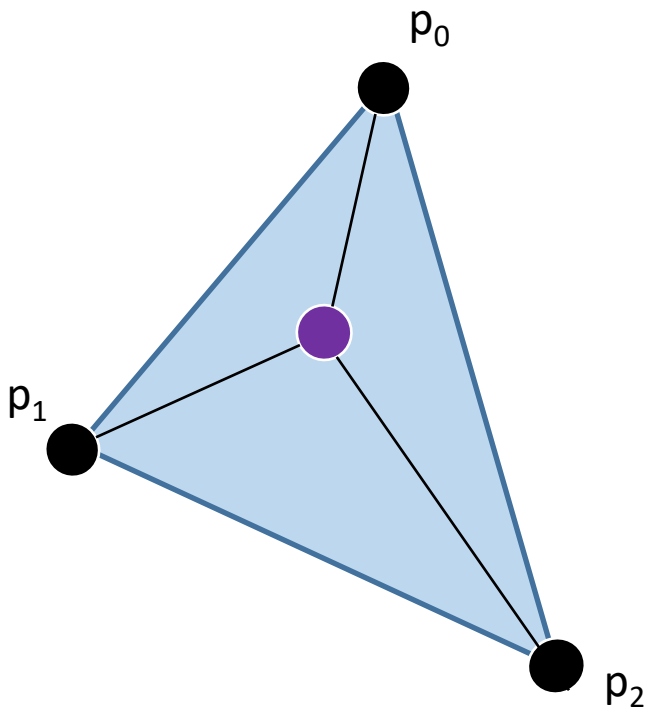
$$\boldsymbol{q} = \alpha \boldsymbol{p}_0 + \beta \boldsymbol{p}_1 + \gamma \boldsymbol{p}_2$$

$$\alpha + \beta + \gamma = 1$$

Inside triangle if:

$$0 \leq \alpha, \beta, \gamma \leq 1$$

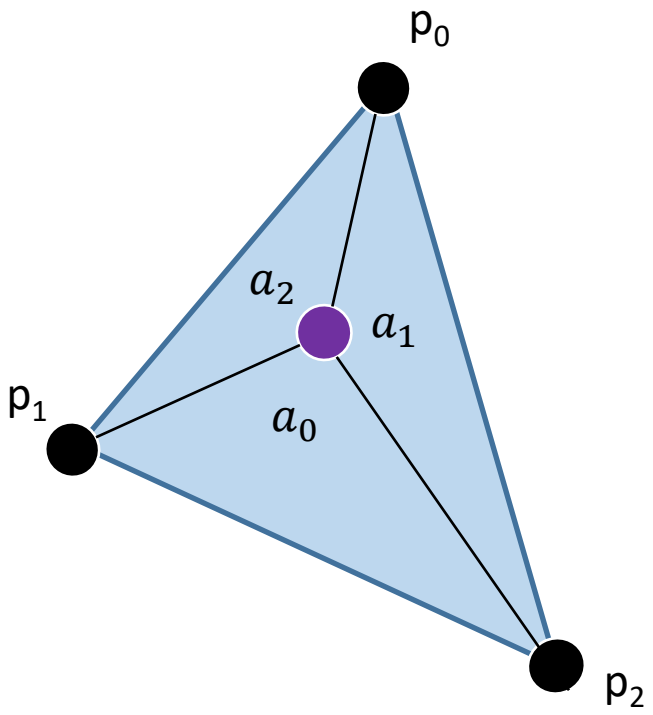
Compute Barycentric Coordinates



$$\boldsymbol{q} = \alpha \boldsymbol{p}_0 + \beta \boldsymbol{p}_1 + \gamma \boldsymbol{p}_2$$

$$\alpha + \beta + \gamma = 1$$

Compute Barycentric Coordinates



$$\boldsymbol{q} = \alpha \boldsymbol{p}_0 + \beta \boldsymbol{p}_1 + \gamma \boldsymbol{p}_2$$

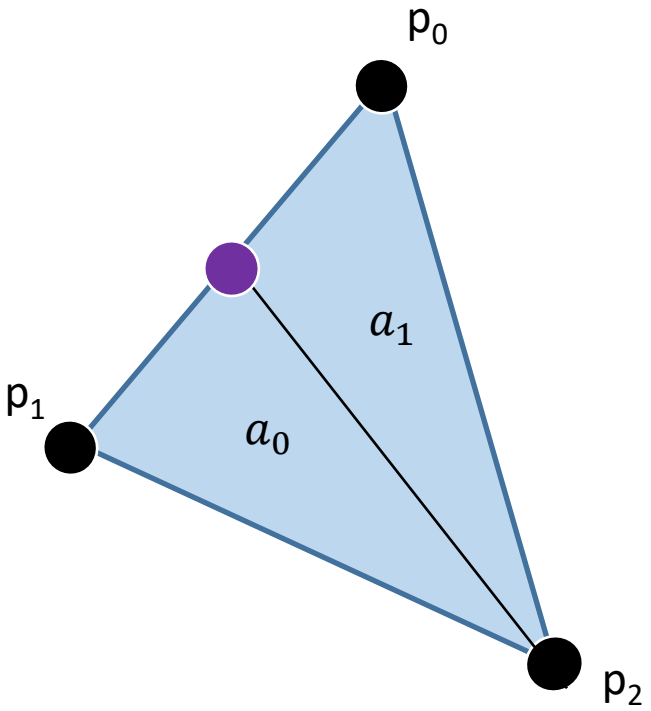
$$\alpha + \beta + \gamma = 1$$

$$\alpha = \frac{a_0}{a} \quad \beta = \frac{a_1}{a} \quad \gamma = \frac{a_2}{a}$$

$$a_0 + a_1 + a_2 = a$$

Compute Barycentric Coordinates

Other computations exist!



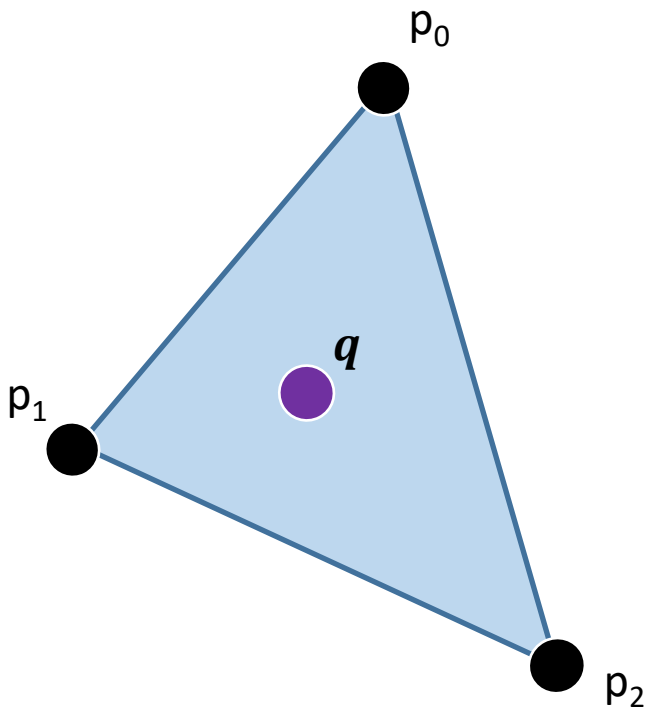
$$\boldsymbol{q} = \alpha \boldsymbol{p}_0 + \beta \boldsymbol{p}_1 + \gamma \boldsymbol{p}_2$$

$$\alpha + \beta + \gamma = 1$$

$$\alpha = \frac{a_0}{a} \quad \beta = \frac{a_1}{a} \quad \gamma = \frac{a_2}{a}$$

$$\alpha = 0.5 \quad \beta = 0.5 \quad \gamma = 0$$

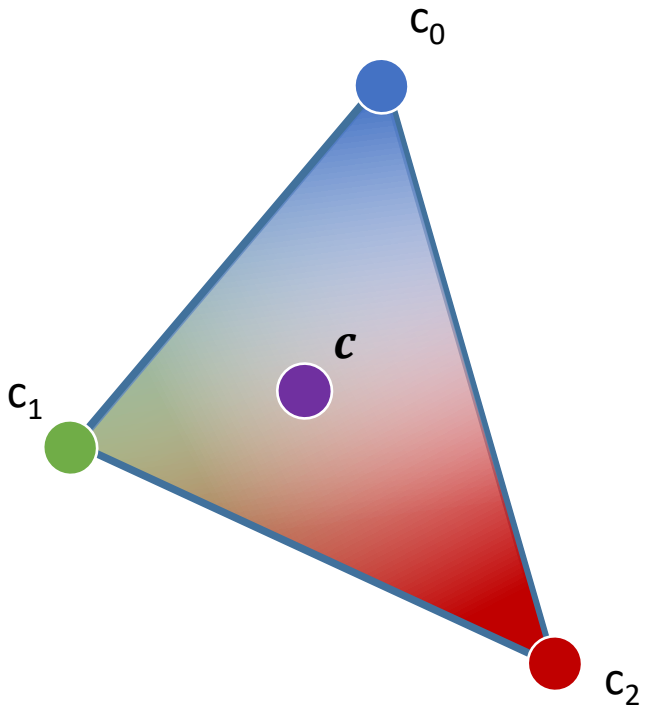
Interpolate Positions



$$\boldsymbol{q} = \alpha \boldsymbol{p}_0 + \beta \boldsymbol{p}_1 + \gamma \boldsymbol{p}_2$$

$$\alpha + \beta + \gamma = 1$$

Interpolate Colors



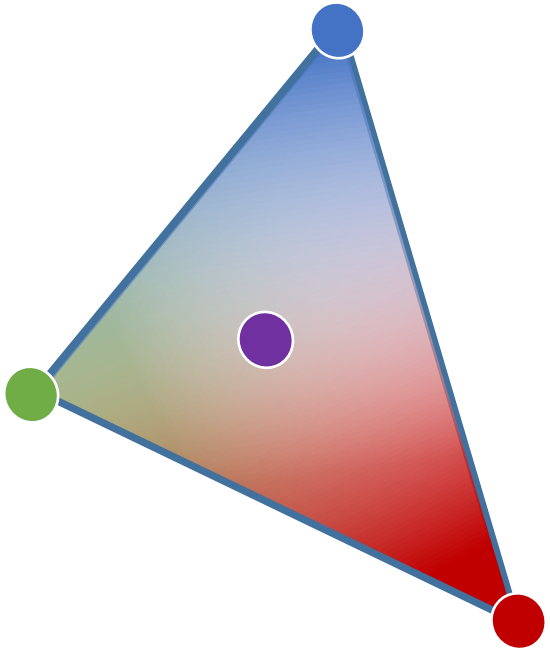
$$\boldsymbol{c} = \alpha \boldsymbol{c}_0 + \beta \boldsymbol{c}_1 + \gamma \boldsymbol{c}_2$$

$$\alpha + \beta + \gamma = 1$$

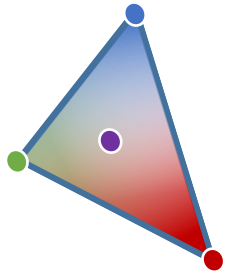
Other interpolated values:

- normals
- texture coordinates

Interpolation in the Prog. Pipeline



Interpolation in the Prog. Pipeline



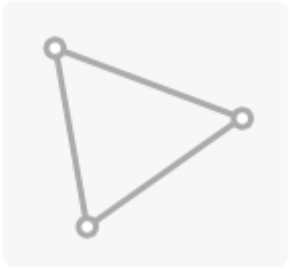
Vertex Processing (Shader)

Rasterizer

Fragment Processing (Shader)



```
v0 = [0.5, 1.0, 0.0]
v1 = [0.0, 0.0, 0.0]
v2 = [1.0, 0.3, 0.0]
```







Triangle Vertices (position buffer)

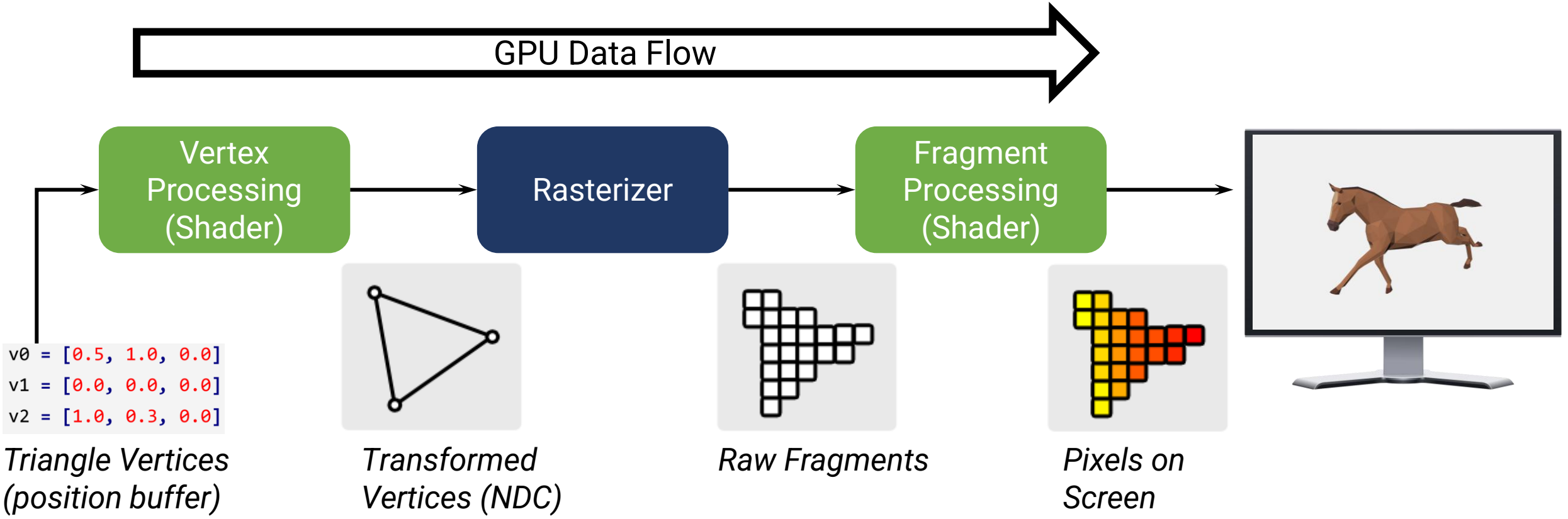
Transformed Vertices (NDC)

Raw Fragments

Pixels on Screen

Back to WebGL

Programmable Pipeline



Setup Shaders: Vertex + Fragment

GLSL Code as String

JS

```
function loadShader(gl, type, source) {
  const shader = gl.createShader(type);

  gl.shaderSource(shader, source);

  gl.compileShader(shader);

  // error handling:
  if (!gl.getShaderParameter(shader, gl.COMPILE_STATUS)) {
    console.error('An error occurred compiling the shaders: ' +
       gl.getShaderInfoLog(shader));

    gl.deleteShader(shader);

    return null;
  }

  return shader;
}
```

https://developer.mozilla.org/en-US/docs/Web/API/WebGL_API/Tutorial/Adding_2D_content_to_a_WebGL_context

Setup Shader Program

JS

```
function initShaderProgram(gl, vsSource, fsSource) {
  const vertexShader = loadShader(gl, gl.VERTEX_SHADER, vsSource);
  const fragmentShader = loadShader(gl, gl.FRAGMENT_SHADER, fsSource);

  // Create the shader program
  const shaderProgram = gl.createProgram();
  gl.attachShader(shaderProgram, vertexShader);
  gl.attachShader(shaderProgram, fragmentShader);
  gl.linkProgram(shaderProgram);

  // some error handling …
  return shaderProgram;
}
```

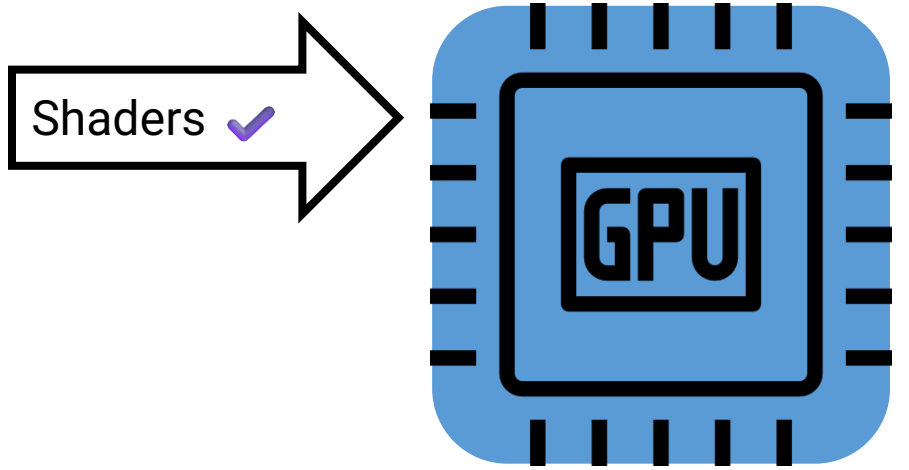
https://developer.mozilla.org/en-US/docs/Web/API/WebGL_API/Tutorial/Adding_2D_content_to_a_WebGL_contex

Use Shader Program

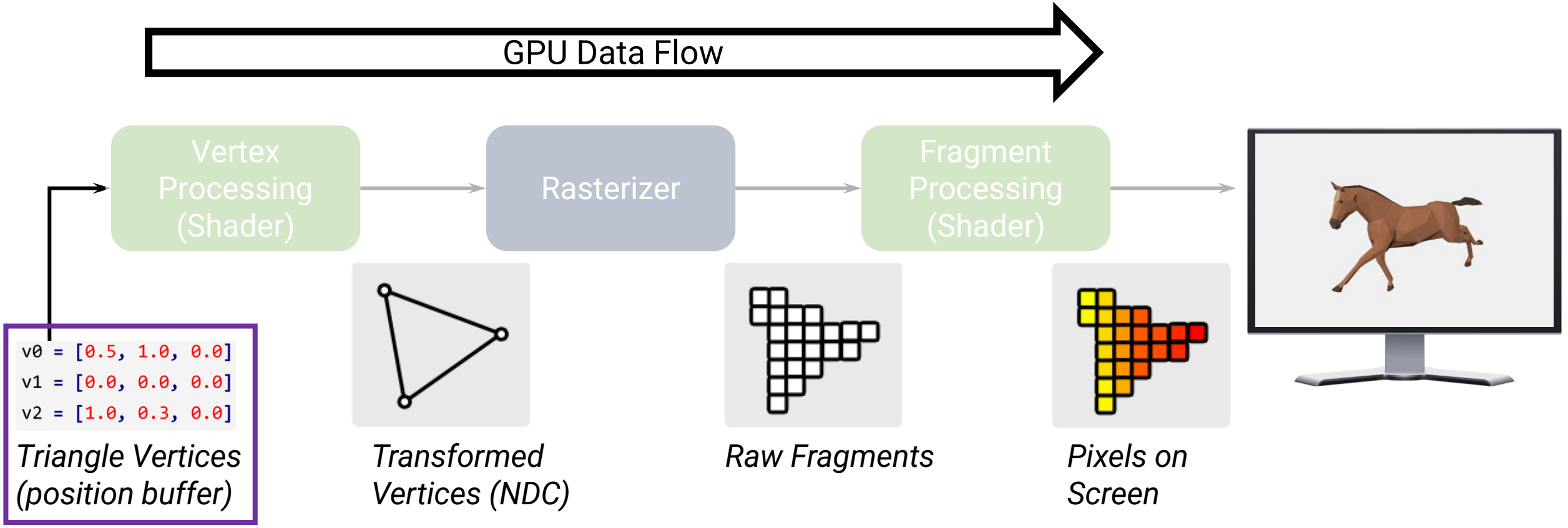
Tell OpenGL to use the shaders:

```
const program = initShaderProgram(gl, vsSource, fsSource);

gl.useProgram(program);
```
JS



Programmable Pipeline

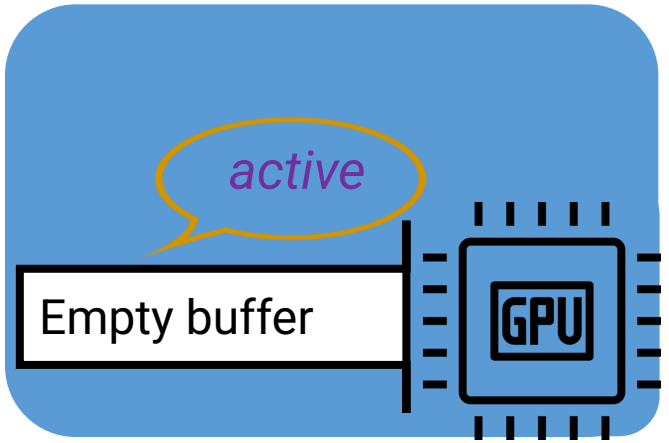


Setup Buffers (Triangle Vertices)

JS

```
const positionBuffer = gl.createBuffer();
gl.bindBuffer(gl.ARRAY_BUFFER, positionBuffer);
const positions = [
    0.5, 1, 0,
      0, 0, 0,
      1, 0.3, 0,
];
gl.bufferData(gl.ARRAY_BUFFER,
      Float32Array.from(positions),
      gl.STATIC_DRAW);
```

x	y	z	x	y	z	x	y	z



Vertex Shader Input

- `in` variables (called attributes) access GPU buffers in our (vertex) shaders.
- We need to tell OpenGL
  - which buffers to use, and
  - how to pull data out of the buffer(s).

vertex positions

Vertex-Shader **GLSL**

```
#version 300 es

layout(location=0) in vec3 position; // 3D Vector

uniform mat4 modelMatrix; // 4x4 Matrix

void main() {
  gl_Position = modelMatrix * vec4( position, 1.0 ); // compute NDC
}
```

Use a Vertex Buffer

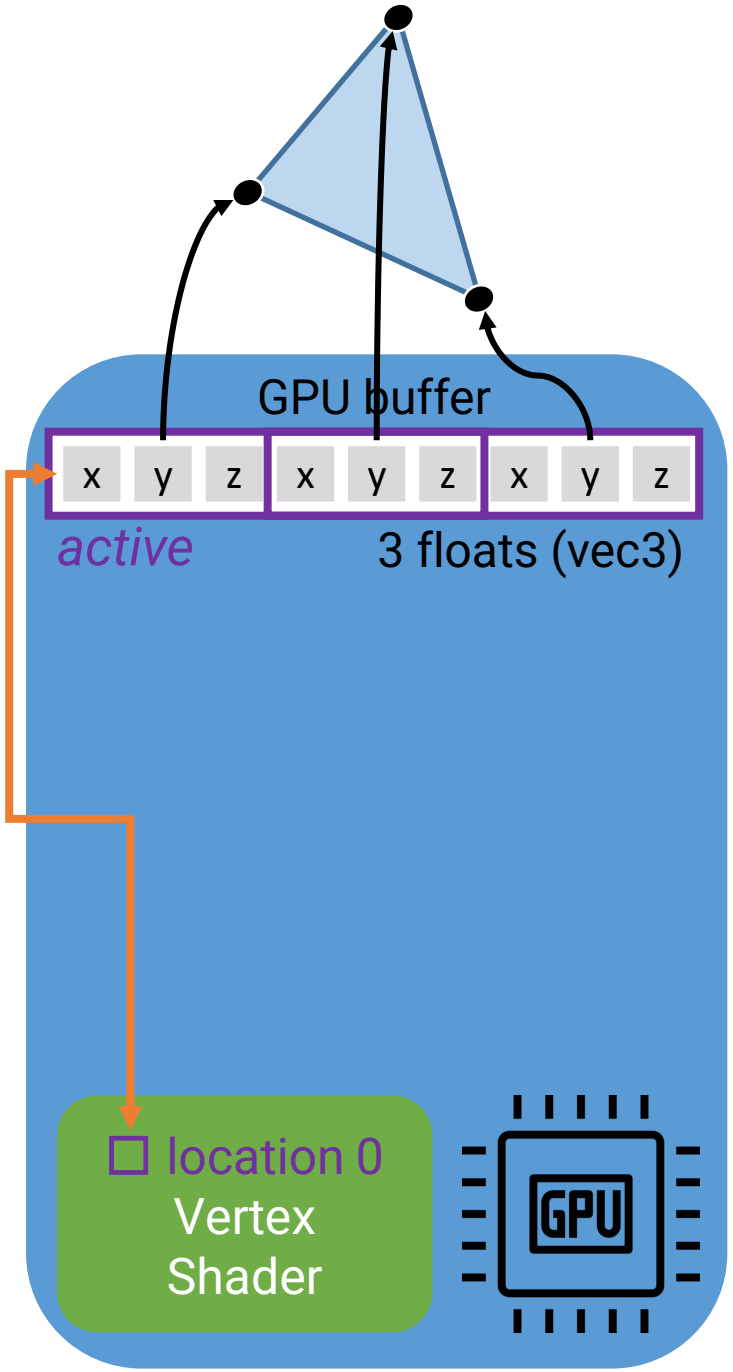
JS

```
gl.bindBuffer(gl.ARRAY_BUFFER, positionBuffer);
const positionLocation = 0; // layout(location=0) in shader!
gl.enableVertexAttribArray(positionLocation);
gl.vertexAttribPointer(positionLocation,
        3,
        gl.FLOAT,
        false, 0, 0);
```

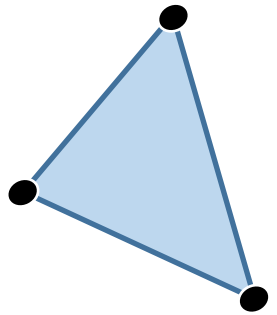
Vertex-Shader **GLSL**

enabled

```
#version 300 es
layout(location=0) in vec3 position; // 3D Vector
uniform mat4 modelMatrix; // 4x4 Matrix
void main() {
  gl_Position = modelMatrix * vec4( position, 1.0 );
}
```



Use Multiple Vertex Buffers



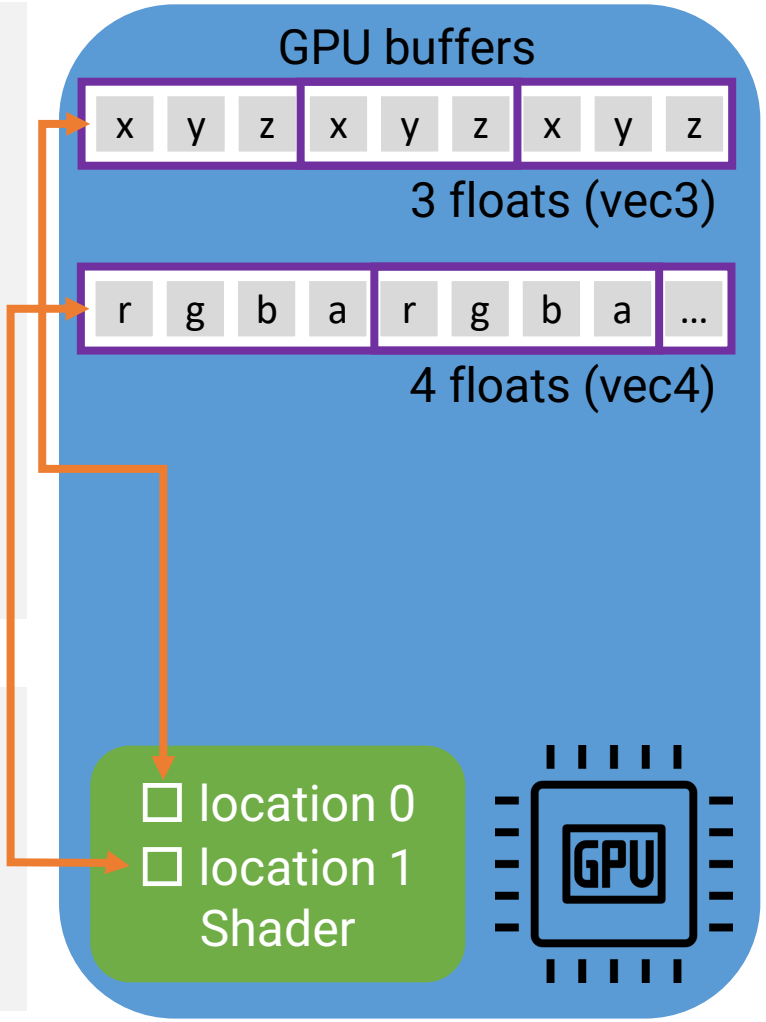
JS

```
gl.bindBuffer(gl.ARRAY_BUFFER, positionBuffer);
gl.enableVertexAttribArray(0);
gl.vertexAttribPointer(0,
        3, gl.FLOAT, false, 0, 0);

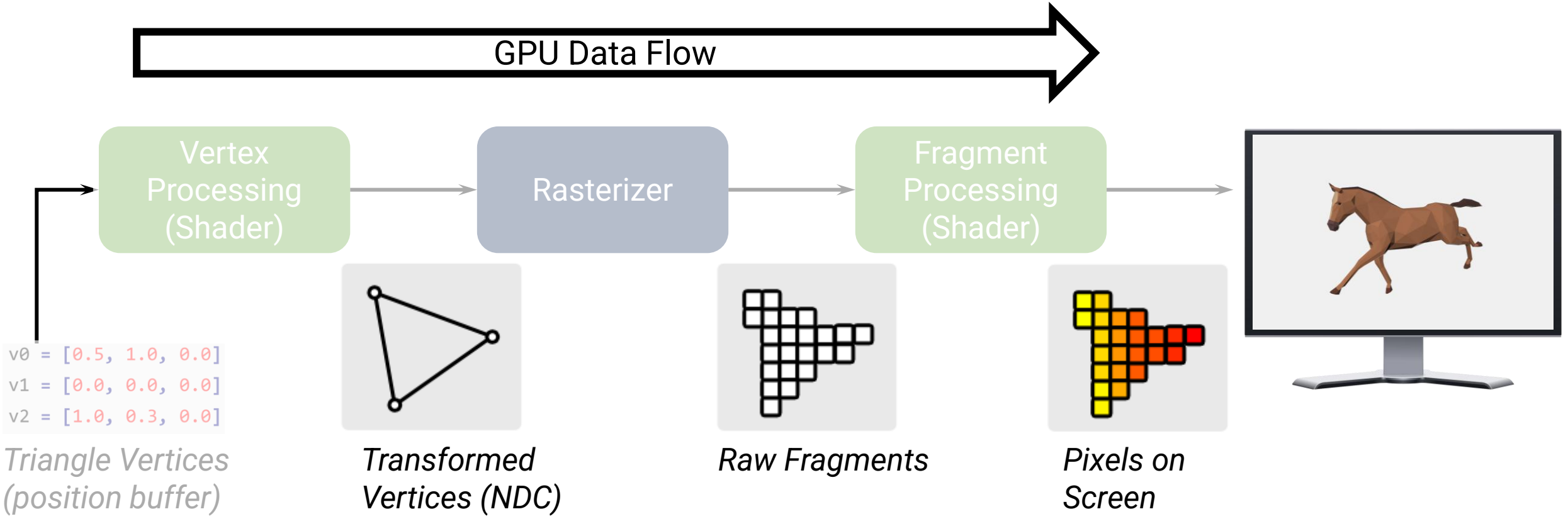

gl.bindBuffer(gl.ARRAY_BUFFER, colorBuffer);
gl.enableVertexAttribArray(1);
gl.vertexAttribPointer(1,
        4, gl.FLOAT, false, 0, 0);
```

Vertex-Shader **GLSL**

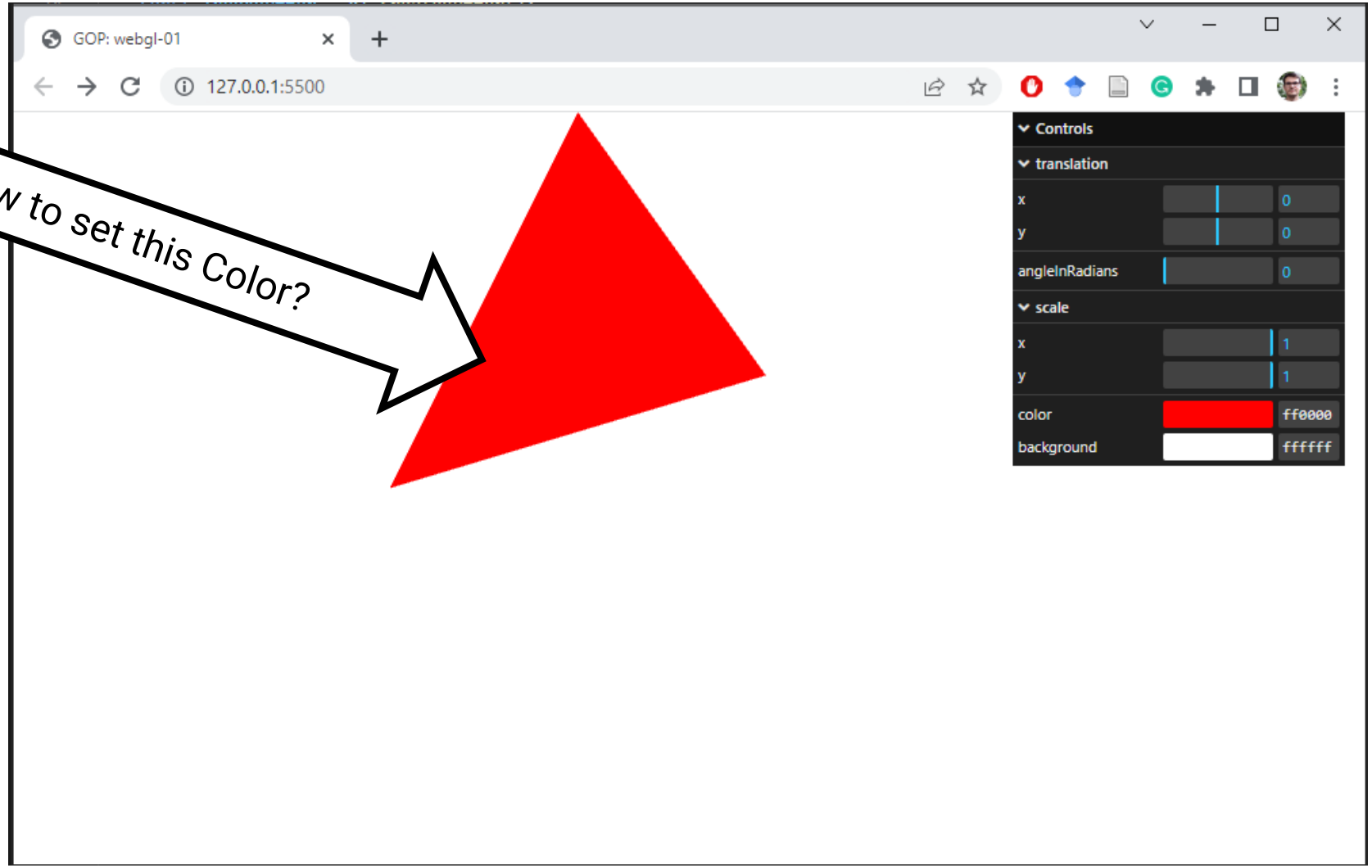
```
#version 300 es
layout(location=0) in vec3 position; // 3D Vector
layout(location=1) in vec4 color; // 4D Vector
…
```



Programmable Pipeline

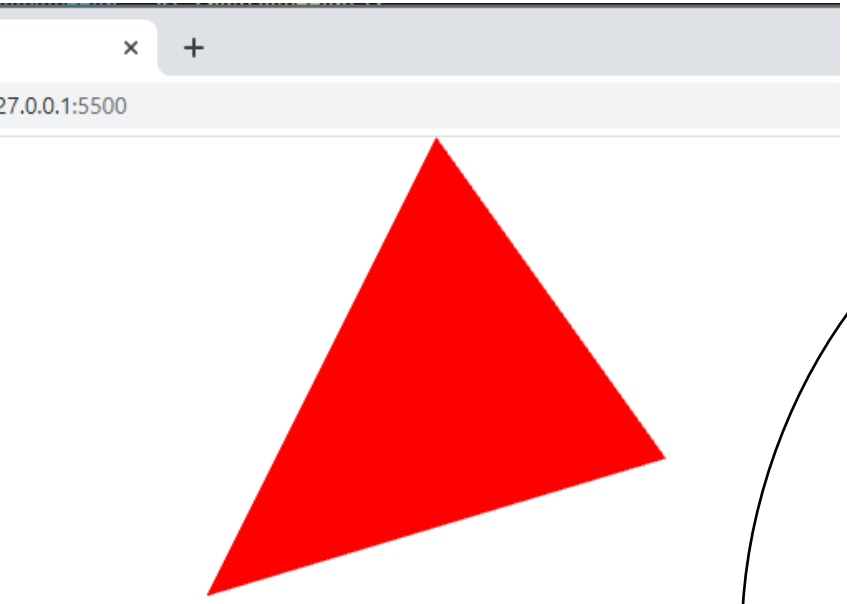


Global Properties



- Uniform for entire triangle
- Attribute (in) would waste a lot of memory!

Uniforms (global properties)



Types must match!

Fragment-Shader **GLSL**

```
#version 300 es
precision highp float;
uniform vec3 color; // color input
out vec4 fragColor; // fragment color
void main() {
    fragColor = vec4( color, 1.0 );
}
```

JS

```
const colorLocation = gl.getUniformLocation(
        program, // WebGL program
        "color"  // variable name in shader
    );
if (colorLocation === null) { /* Error */ }
gl.uniform3fv(colorLocation, [1,0,0] );
```

Setting Uniforms

- Various methods for different data types:
  - Scalar and Vectors: `gl.uniform[1234][uif][v]()`
  - Matrices: `gl.uniformMatrix[234]fv()`

- Examples:

```JS
gl.uniform3f(location, v0, v1, v2);
gl.uniform3fv(location, [v0, v1, v2]);

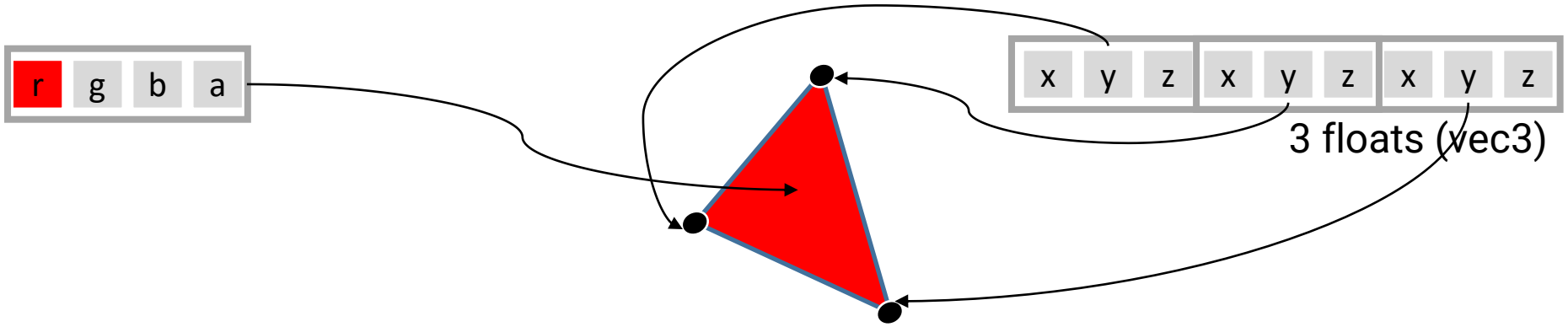

gl.uniformMatrix2fv(loc, false, [2,1, 2,2]);
```

- See:
  - https://developer.mozilla.org/en-US/docs/Web/API/WebGL2RenderingContext/uniform
  - https://developer.mozilla.org/en-US/docs/Web/API/WebGLRenderingContext/uniformMatrix

Uniforms vs. Attributes

Uniforms

Attributes (`in` variables) / Buffers



per primitive (triangle, mesh, ..)

global property

per vertex (local) data

Uniform or Attribute:

Think about any 3D model in Maya or Blender …

Where would you put the following:

- transformation (location, rotation, scale)
- colors
- normals
- lights
- material parameters
- texture coordinates
- camera
- textures
- vertex positions





In & Out Variables

Vertex Shader

- Uniforms are accessible.
- In variables are attributes.
- Out variables are forwarded (&interpolated) to fragments.

```
#version 300 es
layout(location=0) in vec3 position;
uniform vec3 color;
out vec4 new_color;
void main() {
  new_color = …
}
```

Vertex-Shader **GLSL**

Fragment Shader

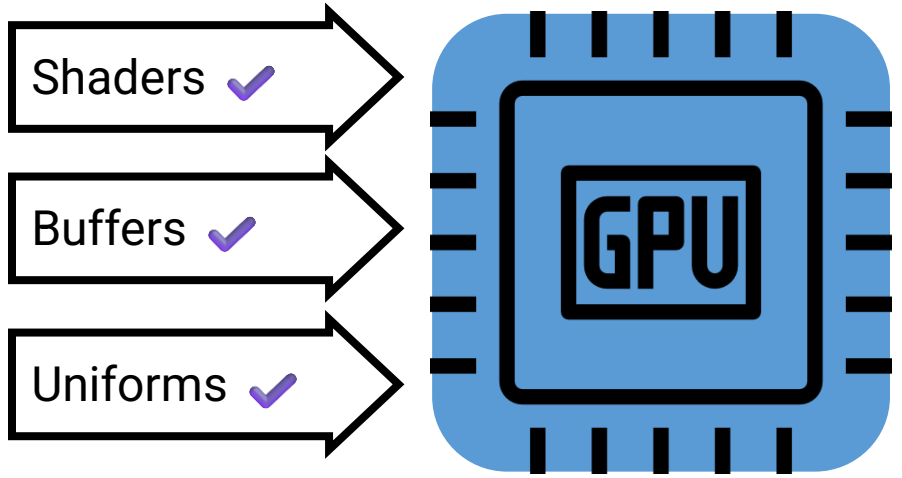
- Uniforms are accessible.
- In variables from vertex shader.
- Out variable is the *Fragment Color.*

```
uniform vec3 color;
in vec4 new_color;
out vec4 fragColor;
void main() {
    fragColor = new_color;
}
```

Fragment-Shader **GLSL**



On the GPU (so far)



We are ready to tell OpenGL to draw the triangles (i.e., start the pipeline):

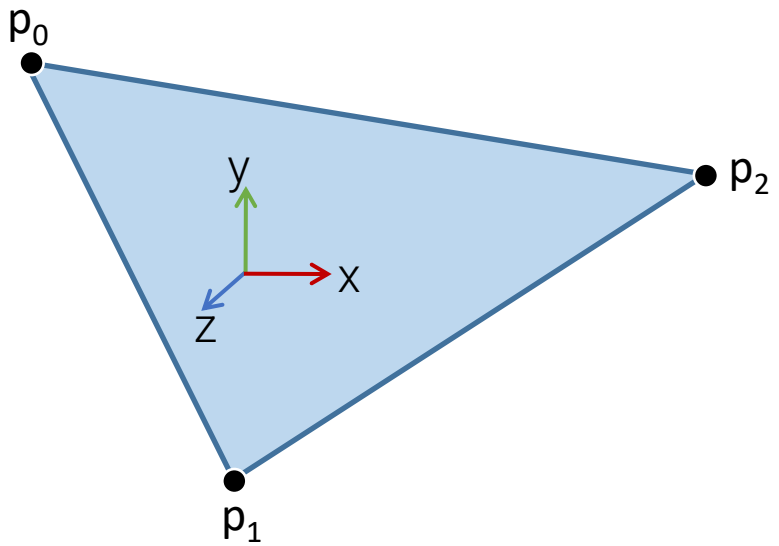
```
gl.drawArrays(gl.TRIANGLES, 0, 3);
```
JS

Number of vertices must match!

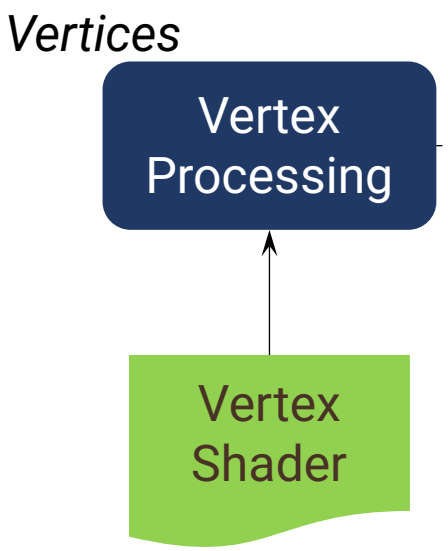
GPU buffer | x | y | z | x | y | z | x | y | z |

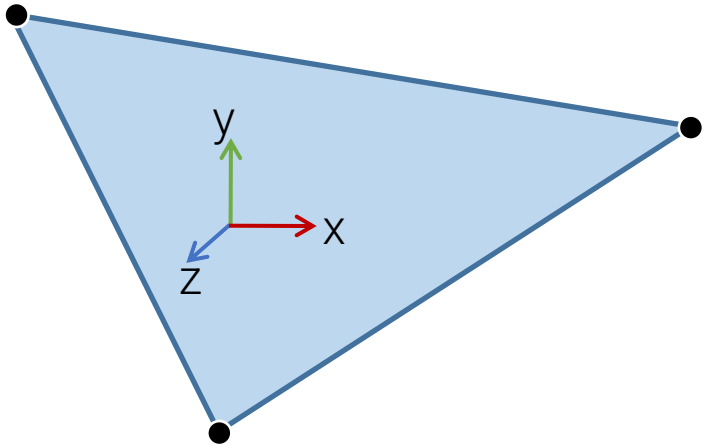
3 x 3 floats (vec3)

Recap Rasterization

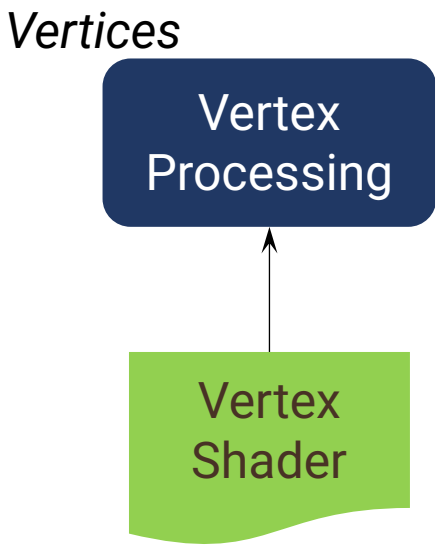


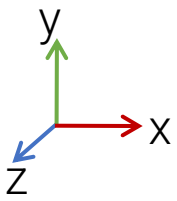
Recap Rasterization

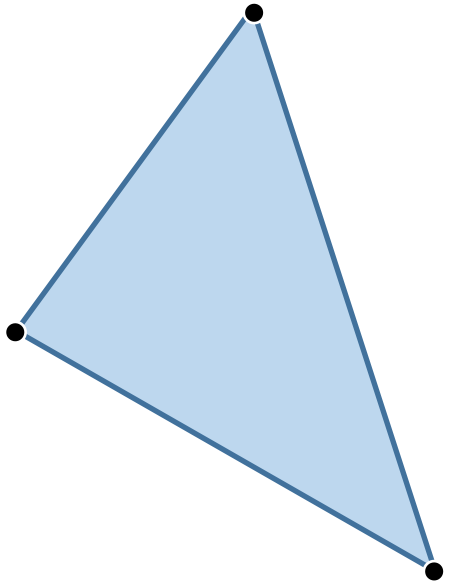




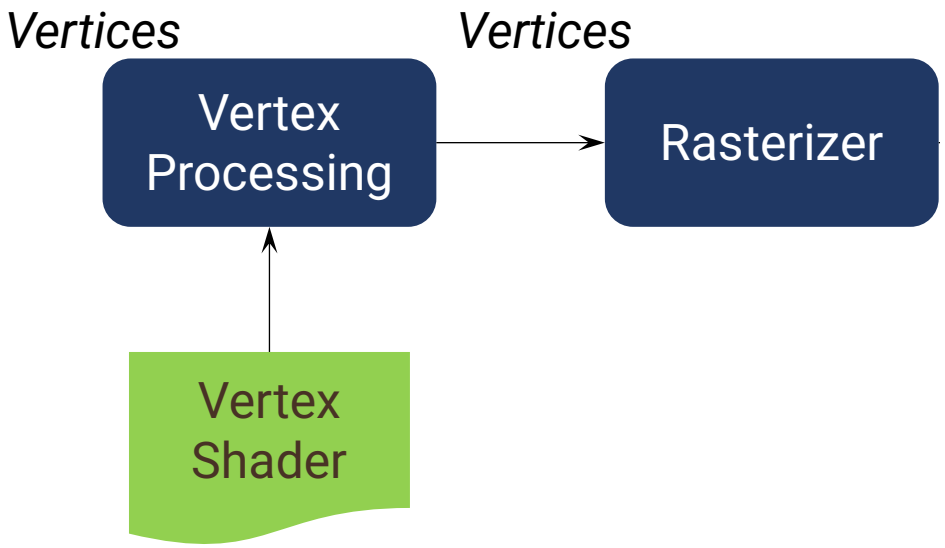
Recap Rasterization

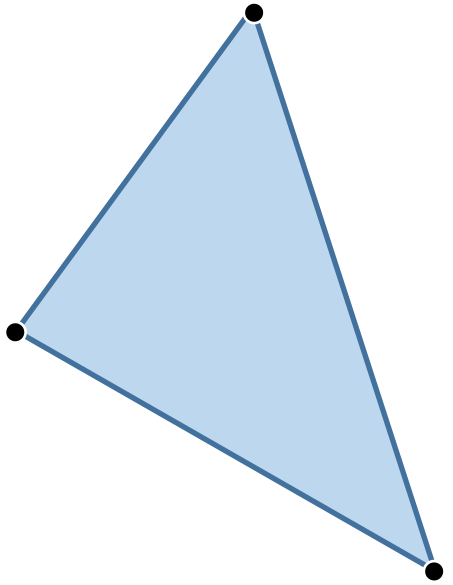




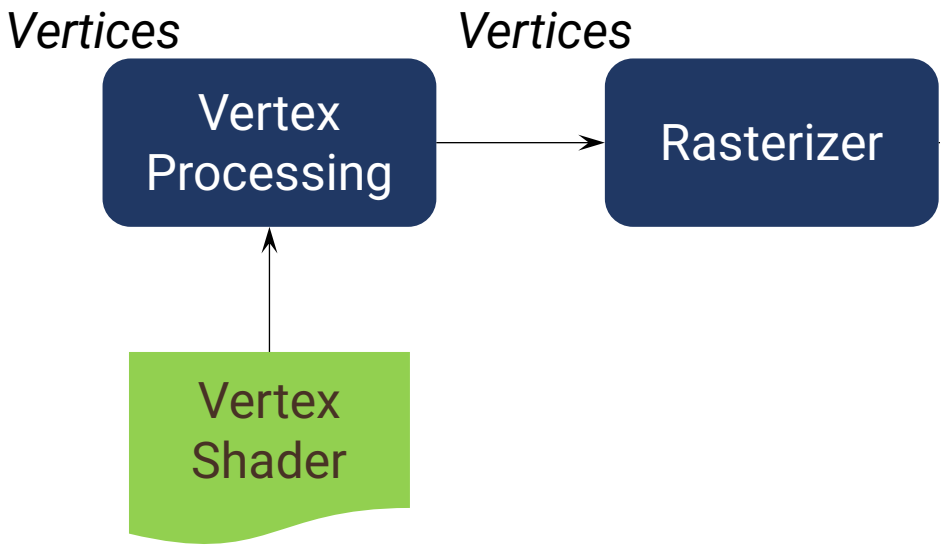


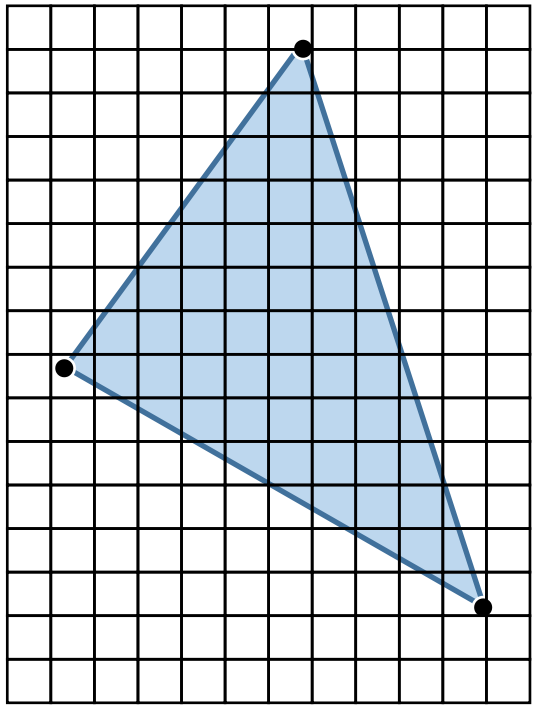
Recap Rasterization



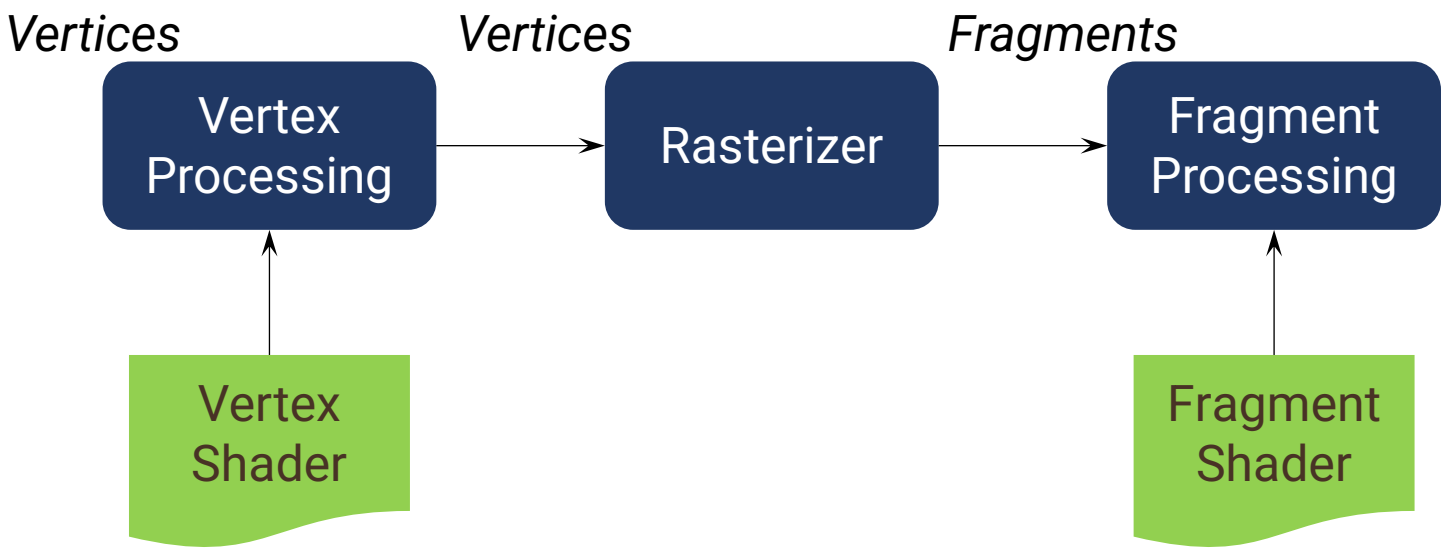


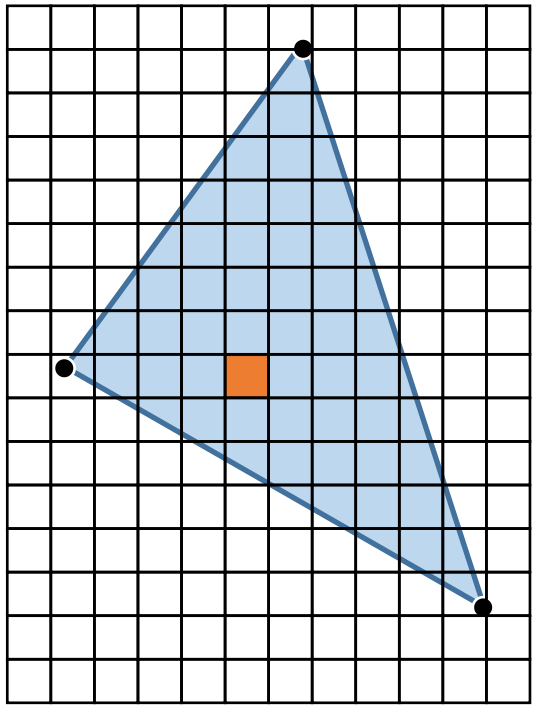
Recap Rasterization



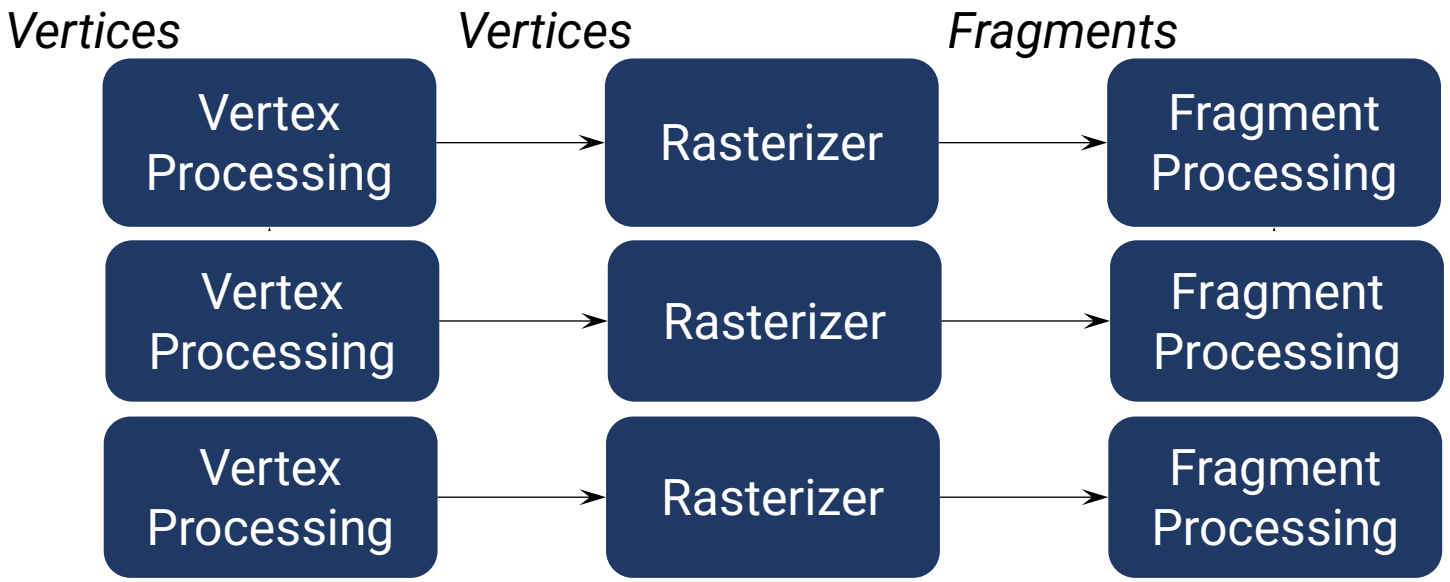


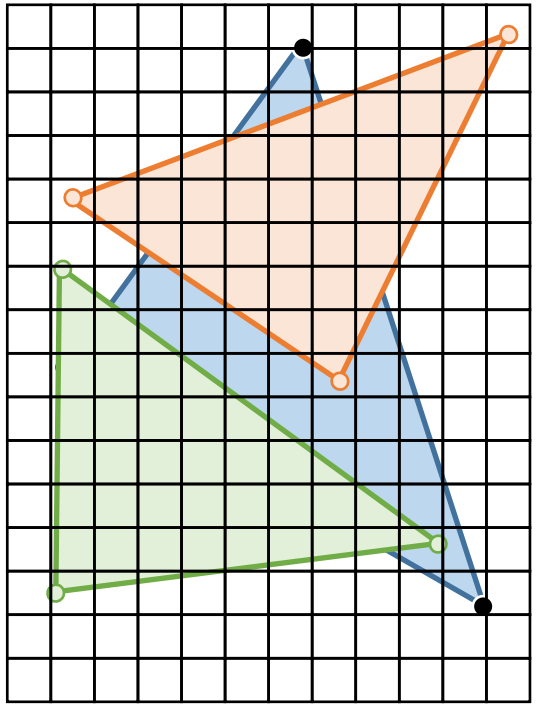
Recap Rasterization





Recap Rasterization

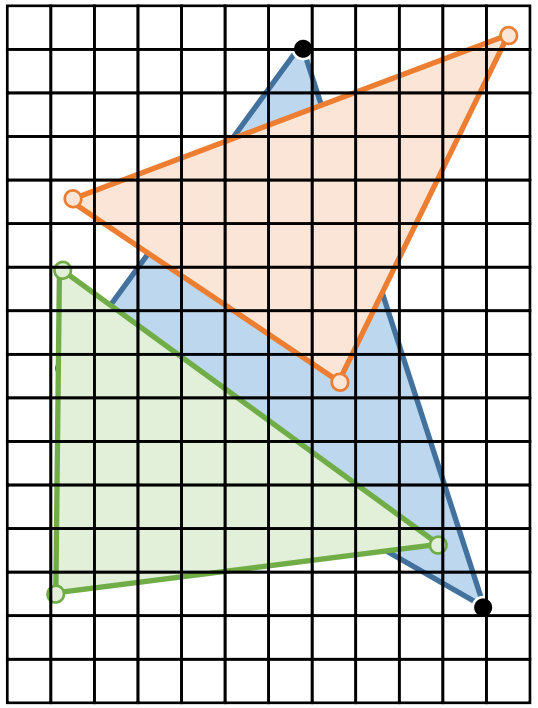




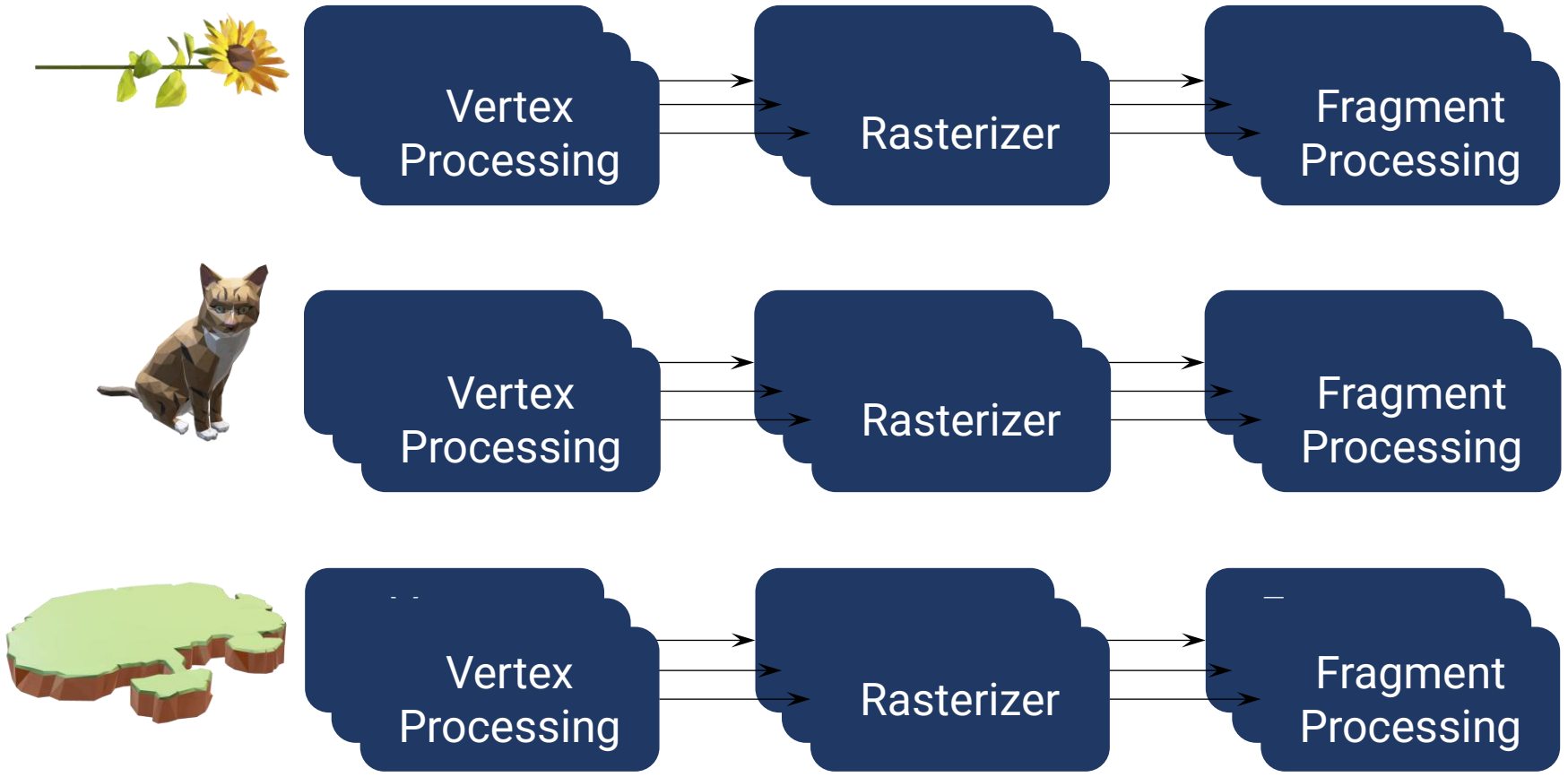
Recap Rasterization

- Rasterization pipeline runs
  - parallel (vertices and fragments) and
  - independent for every mesh/triangle.

- **No** information about the rest of the scene.
  → perfect for GPU

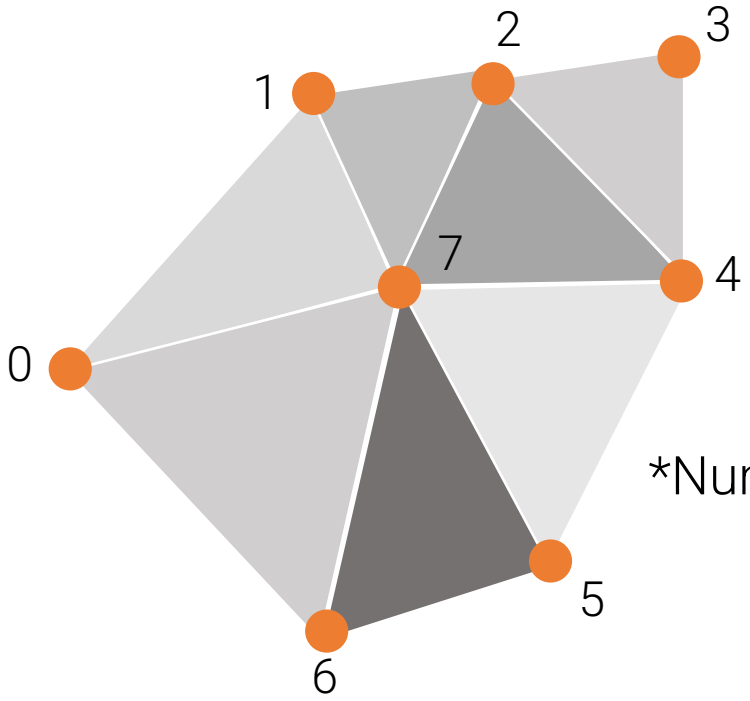


„Draw Calls”


Vertex Processing
Rasterizer
Fragment Processing
Vertex Processing
Rasterizer
Fragment Processing
Vertex Processing
Rasterizer
Fragment Processing

Reusing Vertices: Indices

- Vertices can be re-used
  - Only store and process vertices once
- In addition to an vertex array a list with indices is used

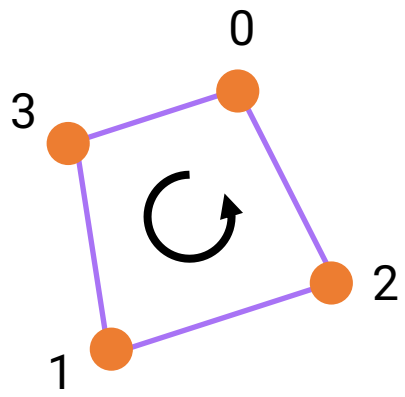


1 triangle

indices	0	7	1	1	7	2	2	7	4	2	4	3	7	5	4	7	6	5	0	6	7
positions	0	1	2	3	4	5	6	7													

*Numbers indicate the index of the vertex, now

Quad



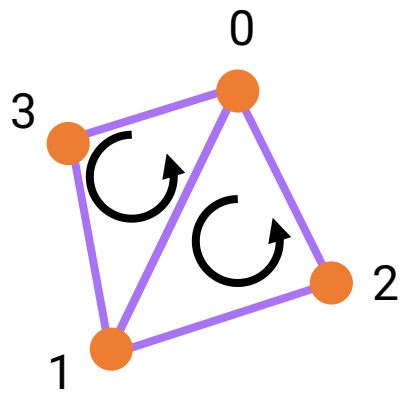
quad

```
v0 = [0.5, 1.0, 0.0]
v1 = [0.0, 0.0, 0.0]
v2 = [1.0, 0.3, 0.0]
v3 = [-.5, 0.7, 0.0]

quad = [v0,v3,v1,v2]
```



Quad with 2 Triangles



quad

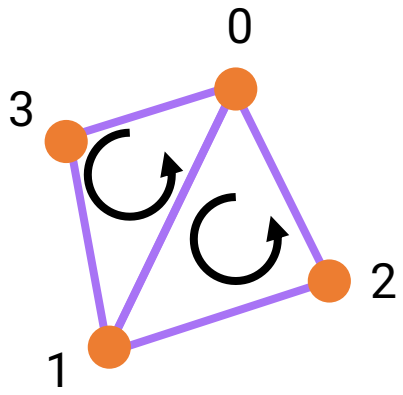
Better!

```
v0 = [0.5, 1.0, 0.0]
v1 = [0.0, 0.0, 0.0]
v2 = [1.0, 0.3, 0.0]
v3 = [-.5, 0.7, 0.0]

# Option 1 - store as 6 vertices:
triangles = [   v0,v1,v2,
                v0,v3,v1 ]

# Option 2 - store 4 vertices:
vertices = [v0,v1,v2,v3]
indices  = [0,1,2,  # triangle 1
            0,3,1]  # triangle 2
```

OBJ file format



quad

Starts with 1!

```
# List of vertices, with
(x y z [w=1]) coordinates.
v 0.5 1.0 0.0
v 0.0 0.0 0.0
v 1.0 0.3 0.0
v -.5 0.7 0.0
# Polygonal face element
(3 indices for a triangle)
f 1 2 3
f 1 4 2
```

.obj

Setup Index Buffers (Quad Vertices)

JS

```
const positions = [
      0.5, 1, 0,
        0, 0, 0,
        1, 0.3, 0,
      -.5, 0.7, 0,
  ];
gl.bufferData(gl.ARRAY_BUFFER, Float32Array.from(positions), gl.STATIC_DRAW);

const indexBuffer = gl.createBuffer();
gl.bindBuffer(gl.ELEMENT_ARRAY_BUFFER, indexBuffer);
const indices = [
            0, 1, 2,   // first triangle
            2, 1, 3,   // second triangle
        ]);
gl.bufferData(gl.ELEMENT_ARRAY_BUFFER, Uint16Array.from(indices), gl.STATIC_DRAW);
```

attribute type

index type

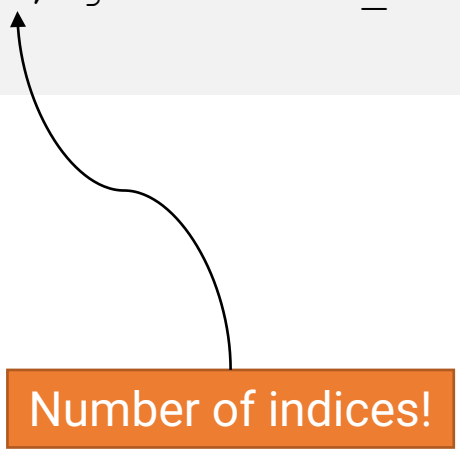
Use and Draw the Index Buffer

JS

```
gl.bindBuffer(gl.ARRAY_BUFFER, positionBuffer);
gl.enableVertexAttribArray(0);// layout(location=0)
gl.vertexAttribPointer(positionLocation, 3, gl.FLOAT, false, 0, 0);

gl.bindBuffer(gl.ELEMENT_ARRAY_BUFFER, indexBuffer);

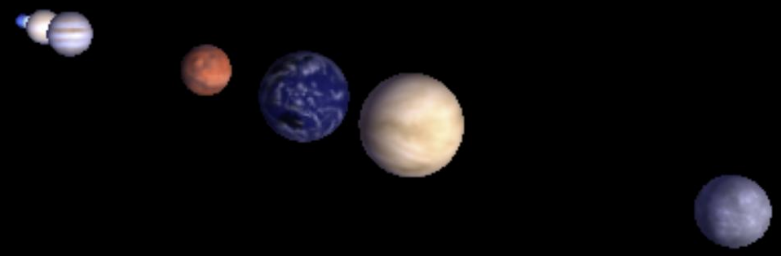
// gl.drawArrays(gl.TRIANGLES, 0, 3);
gl.drawElements(gl.TRIANGLES, 6, gl.UNSIGNED_SHORT, 0);
```



The Bigger Picture

On the GPU:

- Shaders
- Buffers
- Uniforms



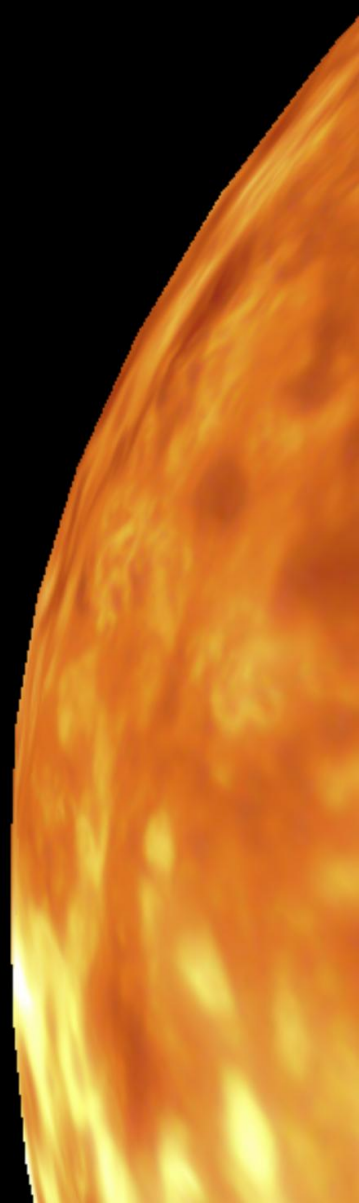
3D Geometries/Objects: Buffers

Appearance: Shaders & Uniforms

Transformation/Animation/Camera: Uniforms

➔WebGL is NOT Object-Oriented 🤔

➔WebGL is only 2D! 🤯



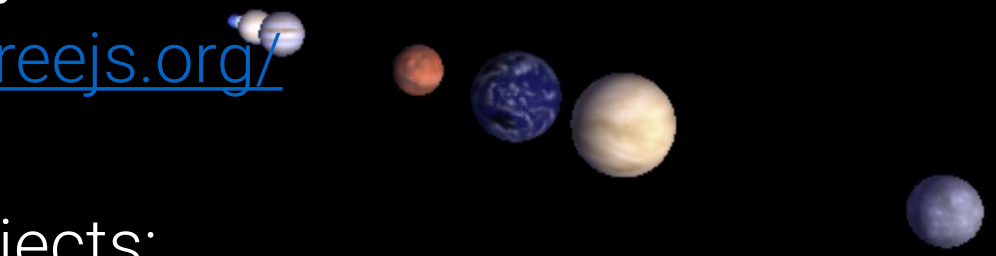
The Entire Course in ~20 Minutes



https://youtu.be/C8YtdC8mxTU?si=xxxVwiDORCGR7tyC

The Labs

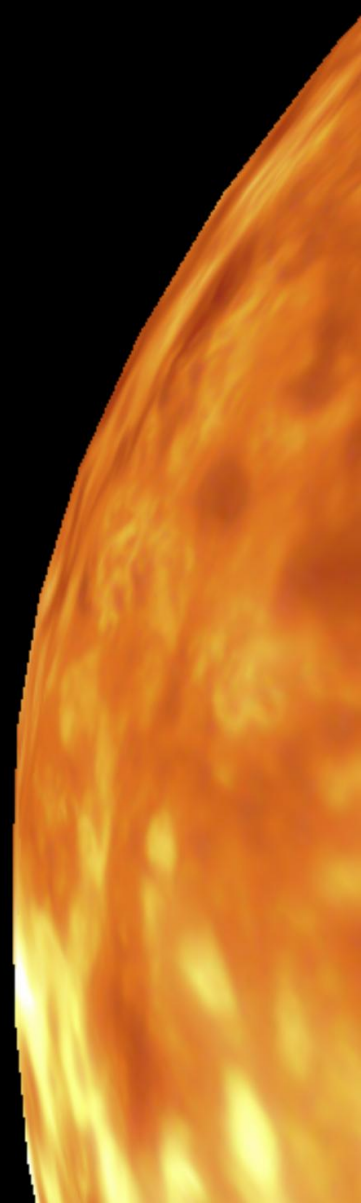
We'll wrap an nice object-oriented framework around!
Inspiration: https://threejs.org/



Scene Graph with Objects:

- WebGLRenderer
- 3D Objects (Transformations & Hierarchies)
  - Mesh <Geometry & Material> (Buffers & Uniforms)
  - Camera (Uniforms)
  - Lights (Uniforms)

➔ We hide WebGL inside these objects.



RTR – Organization

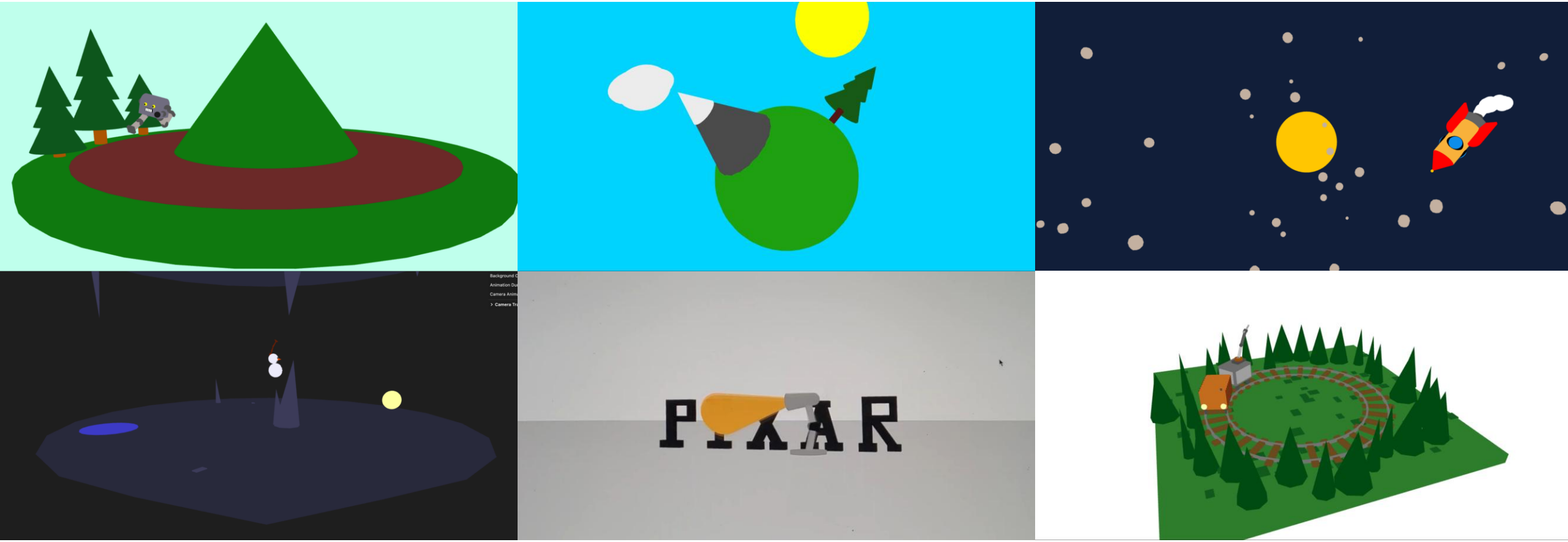
- Lectures:
  - Mostly theory with interactive examples (like today)
  - Grading: exam → you need to pass it

- Labs:
  - Exercises that you solve in the lab and at home
    (simple programming tasks, like small Python scripts)
  - Practically apply the topics of the lecture
  - Grading: 50 % submissions + 50 % written exam (both must be positive)
  - Lecturer: David Schedl (lecture and labs)

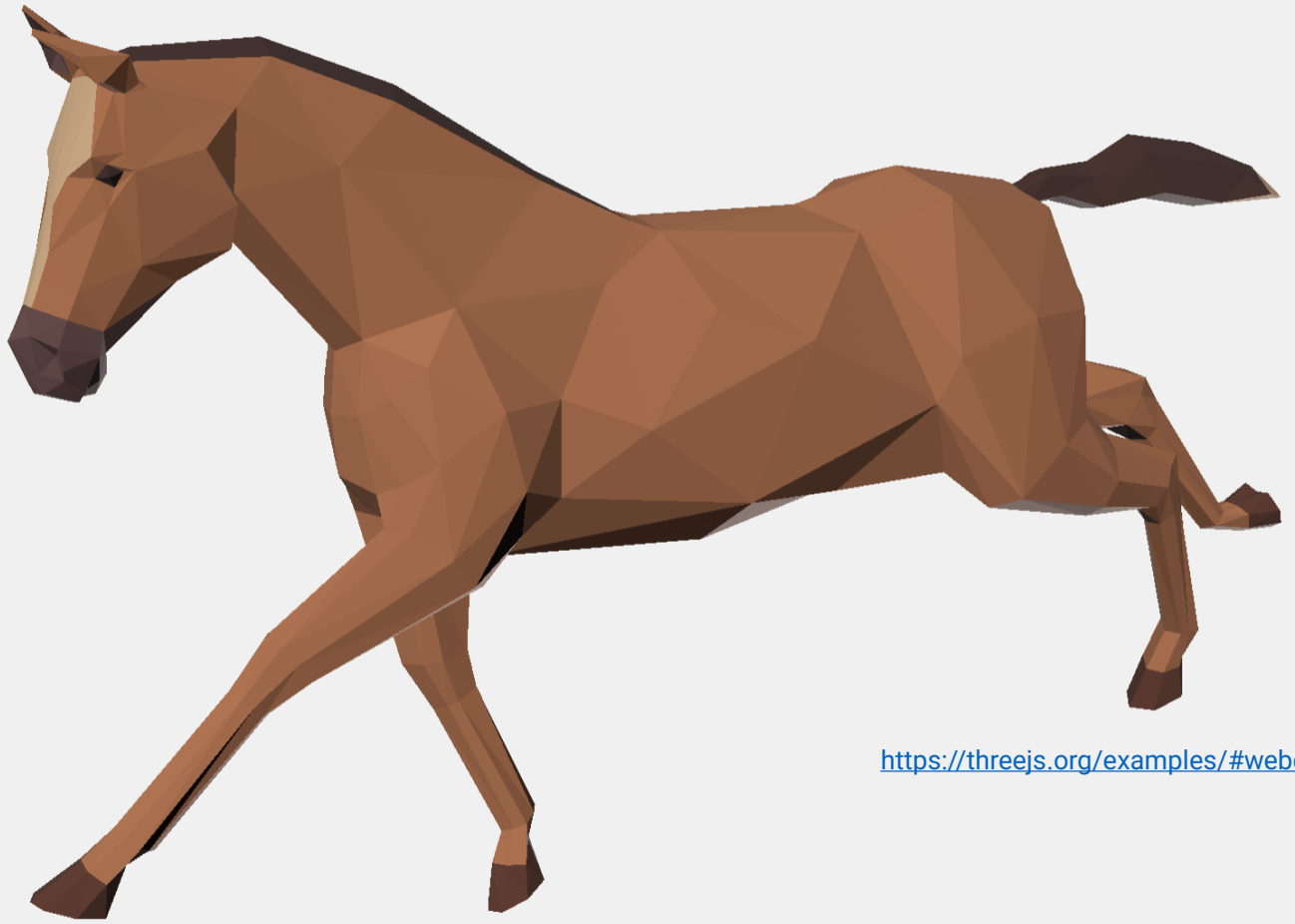
RTR – Organization

- Can get quite complex and confusing (especially if you are struggling with programming)
  - GPU(shader/GLSL) vs CPU(JavaScript)
  - Preparing data (arrays/buffers) for the GPU
  - Two shaders that do different things
  - Parallelism
  - …
- But, it will be fun! ☺

A best-off from last years



Credits/Students: Al Khalily, Kareem Gamal; Dursun Talha Emre; Eckkrammer Jakob; Nowarra Pascal; Rathberger Jonas; Stelzer Jakob



https://threejs.org/examples/#webgl_morphtargets_horse

Questions?

david.schedl@fh-hagenberg.at

